

Modern Bayesian ML theory

Thomas Prior

“La théorie des probabilités n’est que le bon sens réduit au calcul.”

Pierre-Simon Laplace, *Essai philosophique sur les probabilités*, 1840

Abstract

The theoretical part of this work builds up the concepts of Bayesian neural networks from the ground up, starting with the axioms of probability, moving into how Bayes rule can be associated with neural networks, inspecting the various ways of approximating the posterior, and finally introducing a method, CBDL, of generating multiple posteriors to deduce parameter uncertainty in data or compute scarce environments.

The experiments look into various practical approaches to CBDL, the effect of scaling a neural network, and make attempts regarding a specific method of posterior approximation - variational inference.

Glossary

Multi-modal distribution A probability distribution with several local maxima/minima, such that it does not fit one common probability distribution.

Intractable integration An integration difficult to resolve to a simple representation, particularly due to an excessive quantity of variables to integrate over, that has exceed present-day computational capabilities.

Convex set of distributions “A convex set of distributions is a collection of functions such that, if all distributions π_i belong to the set, then the distribution $\sum a_i \pi_i$ belongs to the set (where all a_i are positive and $\sum a_i = 1$).” — Fabio Cozman [Coz97]

A set of probability distributions generated by combining a subset of probability distributions in different ratios.

Highly non-convex distribution A distribution that cannot be easily recreated by merging other common distributions.

Mean-Field Variational Bayes (MFVB) Approximating a posterior distribution by assigning to the neural network parameters θ a set of independent probability distributions such that they can be multiplied to generate a new probability distribution.

Finitely Generated Credal Set (FGCS) A convex set of distributions, generated from an initial set of extremes of the target set, the generation involving continuous or discrete proportions of each.

Credal Bayesian Deep Learning (CBDL) An approach to measuring uncertainty via a set of trained Bayesian neural networks.

Neural network A composition of functions, whereby parameters are adjusted to replicate patterns within data input during an adjustment (training) phase.

1 Introduction

Bayesian Neural Networks seek to solve the problem of neural networks, which are being increasingly relied upon by the general population, making wrong predictions without indication. An original, fictional murder mystery can quickly cause modern LLMs (which feature neural networks as a core component) to produce a wide variety of creative accusations, and provide explicitly requested confidence scores beyond 50%, and this reaction may be detrimental in more consequential fields such as healthcare, engineering, law.

Bayesian Neural Networks attempt to offer a solution by considering a wider variety of initial possibilities when processing data, which currently makes them significantly more computationally intensive (though this may change in the future), but as a result are able to provide more dependable confidence scores to predictions made.

Expected prerequisite knowledge regarding Machine Learning:

- Understanding of what a neural network architecture and weights are
- Understanding of loss functions & training of deterministic neural networks via backpropagation

The assumption is that the reader arrives at Bayesian Neural Networks from a background of non-Bayesian Neural Networks. The title of this thesis features an abbreviation, to imply this.

Some research papers immediately take a deductive, macro approach to introduce the concepts of this field. However, to be more immediately accessible, the following thesis approaches this topic from a micro perspective, and gradually transitions to a macro perspective. As such, it is aimed to be possible to read from start-to-finish, without obliging the reader to do further research to comprehend the concepts (with the potential exception of the prerequisite knowledge).

2 Theory

2.1 Intro to Bayesian statistics

Bayesian statistics is a field of statistics in which attempts to iteratively adjust ideas, and explore a wide range of possibilities, of what the underlying data distribution could be. This is in contrast to what is known as classical statistics, which makes presumptions about the underlying data distribution, and then draws conclusions from it, without the capability of adjusting these presumptions. Ideas of Bayesian statistics emerged in Europe around the 1700s and 1800s. The core idea is Bayes rule, which is derived from fundamental, widely accepted axioms of probability theory. Within the probability space $(\Omega, \mathcal{F}, P)$, where $A, B \in \mathcal{F}$:

Axiom 1 (Non-negative, real probabilities)

$$P(A) \geq 0, \quad \forall A \in \mathcal{F}$$

Axiom 2 (Quantification of full probability space)

$$P(\Omega) = 1$$

Axiom 3 (Countable additivity)

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i)$$

Axiom 4 (Conditional probability)

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

Axiom 5 (Product of independent events)

$$P(A \cap B) = P(A) \cdot P(B)$$

Theorem 1 (Bayes' Rule)

$$P(Hypothesis | Data) = \frac{P(Data | Hypothesis) \cdot P(Hypothesis)}{P(Data)} \quad (1)$$

2.1.1 Understanding Bayes' Rule components

Example 1 (Bayes' Rule rudimentary application)

$$P(fire | alarm) = \frac{P(alarm | fire) \cdot P(fire)}{P(alarm)}$$

Posterior $P(\text{hypothesis} | \text{data})$ the hypothesis given the data, known as the posterior, being what the statistician is looking to deduce, the data usually being the data available that is initially thought not to be entirely independent of the hypothesis.

Likelihood $P(\text{data} | \text{hypothesis})$ is described as the likelihood, the probability that the data would be produced based on the hypothesis being true.

Prior $P(\text{hypothesis})$ is described as the prior, as in, the prior belief before data is collected. In Bayesian statistics, this is modelled as a distribution, which does not have to be symmetric. A common approach is to set this as the same value for all possible hypotheses in the possibility space, and this is known as a uniform prior; a uniform probability distribution.

Evidence $P(\text{data})$ is described as the evidence, and represents the probability of the data being produced regardless of whether the hypothesis is true or not. For now consider example 1, but this component is explained in more detail in part 2.2.1.2.

Note that the evidence, $P(\text{data})$, is sometimes omitted in favour of the equality symbol being replaced with a proportion symbol, resulting in an unnormalised probability (the area under the curve is not equal to 1), but this involves more complications to draw conclusions from.

2.1.2 Applying Bayes' rule

Bayes rule can be approached iteratively, in which case the posterior, $P(\text{hypothesis} \mid \text{data})$, is iteratively swapped with the prior belief, $P(\text{hypothesis})$, upon each data update (i.e. the initial belief gradually changing as new data becomes available). Alternatively, Bayesian statistics can consider a wide range of priors simultaneously, and as a result, the posterior, $P(\text{hypothesis} \mid \text{data})$ will generate a variety of points which make up a probability distribution, which may be significantly less symmetric and better representative than a typical classical statistics model, whilst still providing confidence intervals to conclusions.

2.2 Neural networks

A neural network is a composition of functions, typically made up of linear regression and ReLU-like activation functions, which feature a large quantity of parameters. A dataset made up of $(\mathbf{x}, \mathbf{y})$ tuples is input into the neural network to make predictions $\hat{\mathbf{y}}$, compared with an expected output via a loss function L , and finally parameters are iterated via differentiation and gradient descent, then this process is repeated such that the predictions $\hat{\mathbf{y}}$ become very close to the expected output $\mathbf{y}$. One instance of all parameters within a neural network is represented by $\boldsymbol{\theta}_i$, whilst each individual parameter within $\boldsymbol{\theta}_i$ may be represented as $\theta_{i,j}$. A neural network may have (uncountably) infinite possible instances of $\boldsymbol{\theta}_i$.

2.3 Intro to Bayes Rule applied to Neural Networks

In contrast to non-Bayesian neural networks, Bayesian neural networks are intended to be trained and utilised in batches. That is to say, multiple models are trained and run simultaneously, and the results of all collated, so as to produce probability distributions.

Note that to call neural networks Bayesian may arguably be, historically, a misnomer; the neural networks involved in Bayesian neural networks operate the same regardless of whether Bayesian concepts are applied to them or not, but Bayesian statistics can be thought of as an approach to their setup, and output, as a set, that provides probabilistic insight. Generally speaking, a larger quantity of neural networks will allow for more significant insights, by providing more predictions.

2.3.1 Bayes' rule applied to Neural Networks

In the case of Bayesian Neural Networks, the components of Bayes rule are interpreted as follows:

Example 2 (Bayes rule with minimalist neural networks parameters)

$$P(\boldsymbol{\theta} \mid D) = \frac{P(D \mid \boldsymbol{\theta}) \cdot P(\boldsymbol{\theta})}{P(D)}$$

2.3.2 Definitions

These definitions will occur frequently throughout this text.

$\boldsymbol{\theta}$ represents the full set of parameters (weights) which can be assigned to one neural network. This neural network may have any possible quantity of parameters, and therefore $\boldsymbol{\theta}$ could be of any dimension. The values within $\boldsymbol{\theta}$ are members of the real numbers. Similarly, to speak of “values of $\boldsymbol{\theta}$ ”, as a plural, means multiple possible sets of $\boldsymbol{\theta}$ that one neural network could take.

$$\boldsymbol{\theta} \in \mathbb{R}^N, N \in \mathbb{N}$$

Note that the set of all possible values of $\boldsymbol{\theta}$, $S_{\boldsymbol{\theta}}$, is (uncountably) infinite due to the above declaration. Nonetheless, when $S_{\boldsymbol{\theta}}$ is graphed, such that the x -axis represents every possible value of $\boldsymbol{\theta}$, there are to be considered constraints when $\boldsymbol{\theta}_i$ is iterated to $\boldsymbol{\theta}_{i+1}$, such as by changing one value within the vector at a time, so as to allow for smooth curves. The specifics of how these permutations can be hypothetically arranged neatly, especially as dimensions increase, are omitted.

Note that some papers use w to symbolise parameters of a model, instead of $\boldsymbol{\theta}$, and then use $\boldsymbol{\theta}$ to symbolise parameters of a probability distribution.

Ψ represents one instance of a neural network, which could be made up of any quantity and any arrangement of neurons, hidden layers, connections, and activation functions, unless explicitly specified. Ψ never includes the parameters, which are always specified separately, as a value of $\boldsymbol{\theta}$.

In purely mathematical terms, Ψ is an intricate function made up of the composition of a variety of functions, including activation functions and linear regression functions, the latter being based on values of $\boldsymbol{\theta}$. An instance of a model with parameters that have been assigned to it, such that it can be used as a typical function, is represented as $\Psi_{\boldsymbol{\theta}}$.

D represents the training dataset as a whole.

D_y and D_x represent the elements of the training data, in that the training data would be made up of (x, y) tuples, where x represents one instance of input to the neural network (x may be multidimensional), y represents one corresponding instance of output of the neural network (y may be multidimensional), and therefore D_x represents the full set of inputs within the training data, and D_y the full set of outputs. When D_x and D_y are depicted within the same formula, this means that each instance is matched with its original tuple.

$$x \in \mathbb{R}^X, X \in \mathbb{N}$$

$$y \in \mathbb{R}^Y, Y \in \mathbb{N}$$

Where X is the quantity of neurons in the input layer, and Y is the quantity of neurons in the output layer, of the neural network.

In the case of a neural network that is only used for classification, $\mathbb{R}^Y$ could be replaced with $\mathbb{N}^Y$.

$$D_x \in \mathbb{R}^{I \times X}$$

$$D_y \in \mathbb{R}^{O \times Y}$$

Where I and O each represent the quantity of tuples in the training dataset.

$$D \in \mathbb{R}^T$$

Where T is the quantity of tuples within the training dataset, which would be equal to $IX + OY$.

2.3.3 Interpreting the components of Bayes rule in the context of Bayesian Neural Networks

$\mathbf{P}(\boldsymbol{\theta} \mid \mathbf{D})$, the posterior, is the essence of training a neural network; what is wanted to be known is the probability of different values $\boldsymbol{\theta}$ of the network, based on the (training) data available.

$\mathbf{P}(\boldsymbol{\theta})$, the prior, is the initial belief in what the parameters would most likely be. A uniform prior, as covered in part 2.1.1, may be specified. Alternatively, most conventionally in the realm of Bayesian Neural Networks, a common distribution (e.g. Gaussian) and distribution parameters may be selected, which values of $\boldsymbol{\theta}$ can be sampled from. Based on current developments in conventional/non-Bayesian neural networks, some would expect the values of weights to conform to the limitations of backpropagation [JLB⁺20] (i.e. not small enough such as to cause vanishing gradients when multiplying partial derivatives, but not large enough to cause exploding gradients either). However, there is allegedly no theoretical argument to claim that current conventions are best [JLB⁺20], but frequently a mean of 0 and a standard deviation of 1 can be found.

$\mathbf{P}(\mathbf{D} \mid \boldsymbol{\theta})$, more precisely represented as $P(D_y \mid D_x, \boldsymbol{\theta})$, or alternatively, $P(\Psi_{\boldsymbol{\theta}}(D_x) \mid D_x, \boldsymbol{\theta})$, the likelihood that a given set of model parameters $\boldsymbol{\theta}$ would produce the expected training data output D_y , given the training data D_x is input into the model $\Psi_{\boldsymbol{\theta}}$.

Note that the neural network architecture is a core piece of this component, as a specific model Ψ is associated with different values of $\boldsymbol{\theta}$, each value of $\boldsymbol{\theta}$ having a corresponding value $P(\boldsymbol{\theta})$, which the training data D_x is then input into, for each value of $\boldsymbol{\theta}$. The output is then compared against the training data D_y , such as by associating a distribution with $P(D \mid \boldsymbol{\theta})$, and a numerical probability can be derived.

The typical model is of each pair of input and output tuples (x, y) , $x \in D_x$, $y \in D_y$ are i.i.d, such that the following is true:

$$P(D \mid \theta) = \prod_{(x,y) \in D} p(y \mid x, \theta)$$

Example 3 (Modelling $P(D \mid \theta)$) *In the case of a binary classifier, based on a constant b , the probability that the prediction of the model output $\hat{y}$ is the same as the expected training data output y could be based on the distance from the boundary value b that the point is found.*

Specifically, $P(D \mid \theta)$ can be deduced from a Gaussian distribution CDF, calibrated such that the mean of the Gaussian is on one side of the boundary value, $b < \mu$:

$$\begin{aligned} P(y \mid x, \theta) &= \text{Gaussian}_{CDF}(\Psi_{\theta_a}(x)) \\ &= \frac{1}{\sqrt{2\pi} \sigma} \int_{-\infty}^x \exp\left(-\frac{(t - \mu)^2}{2\sigma^2}\right) dt \end{aligned}$$

]Therefore, $P(\hat{y}_1 \mid x, \theta_a) = \max_{x \in D_x} \Phi_{(\mu, \sigma)}(\Psi_{\theta_a}(x))$ for a value θ_a that results in a prediction y_1 that is of a distance $(b - \mu)$ from the boundary classification line, $P(\hat{y}_0 \mid x_i, \theta_b) \approx 0$ for a value θ_b that results in a prediction y_0 on the opposite side of the boundary line and of significant distance from it.

$P(D)$, the evidence, is the probability that the dataset would occur to begin with. This seems initially ambiguous, in the context of neural networks, but is generally extrapolated to the following, as in Proof 1:

$$P(D) = \int_0^\infty P(D_y \mid D_x, \theta_i) \cdot P(\theta_i) d\theta_i$$

Where θ_i represents all possible values of θ .

Proof 1 (Deriving $P(D)$) *Consider a neural network that operates as an infinite-class classifier, such as via infinite neurons in the output layer, where θ represents one specific set of model parameters.*

$$P(D) = P(D \cap \theta_1) + P(D \cap \theta'_1)$$

Instead of parameter θ_1 not occurring, consider instead the possibility of all alternatives to θ_1 occurring (which would be an infinite set), including values such as $\theta_2, \theta_3, \theta_4, \theta_5$, each with slightly different values, such that no two are identical; $P(D \cap \theta_i)$ can therefore be modelled as a continuous variable.

If the probabilities of all potential values of θ (which would be a multidimensional vector) were then mapped to a graph with a hypothetically infinite x-axis, each increment along the x-axis representing a small change among the elements of θ , the specific probability of a value θ could then be observed, given a function f that can map any given θ to a position of S_θ , provided this function is injective. As a result, the total value of $P(D)$ would be the area under the curve of a graph in which the y-axis represents the probability of $P(D \cap \theta_i)$, and the x-axis represents all possible values of θ .

$$P(D) = P(D \cap \theta_1) + P(D \cap \theta'_1)$$

Where...

$$\begin{aligned} P(D \mid \theta'_1) &= \int_2^\infty P(D \mid \theta) d\theta \\ \therefore P(D \cap \theta_1) + P(D \cap \theta'_1) &= \int_1^\infty P(D \mid \theta) d\theta \\ \therefore P(D) &= \int_1^\infty P(D \mid \theta) d\theta \end{aligned}$$

The above integrand could then be represented as follows, via Axiom 2.

$$\begin{aligned} P(D \mid \theta) &= \frac{P(D \cap \theta)}{P(\theta)} \\ \therefore P(D \cap \theta) &= P(D \mid \theta) \cdot P(\theta) \\ \therefore P(D) &= \int_1^\infty P(D \mid \theta) \cdot P(\theta) d\theta \end{aligned}$$

□

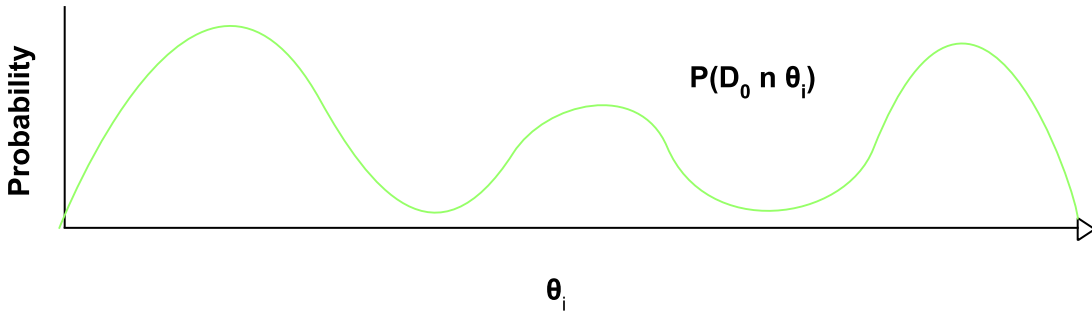


Figure 1: sample graph to show how different values of θ_i could map to a graph, based on one specific instance of training data D_0 and one specific neural network model Ψ . Note that θ_i is multidimensional; this is a 1-dimensional representation for fit.

More simply, the denominator of the posterior, $P(D)$, can be thought of as the (continuous, hence integrand) combination of all the possibilities of the numerator.

2.4 Training Bayesian Neural Networks

During training, the posterior $P(\boldsymbol{\theta} \mid D)$ is developed via the training data and a subset of $S_{\boldsymbol{\theta}}$. Note that this distribution may be highly dimensional (due to the parameters within $\boldsymbol{\theta}$ being multidimensional) and highly non-convex.

Initially, one architecture Ψ must be editorially specified, i.e. the arrangements and quantities of neurons, hidden layers, connections. Next, the values of $\boldsymbol{\theta}$ are to be considered, and the different approaches to assigning them is what constitutes the training process.

Specifically, the architecture Ψ plays an essential role in deriving the likelihood, $P(D_y \mid D_x, \boldsymbol{\theta})$, as an instance of the training data, x , is run through an instance of Ψ with the value $\boldsymbol{\theta}$ used as model parameters, to derive y , $P(y \mid x, \boldsymbol{\theta})$, or, more verbosely, $P(\Psi_{\boldsymbol{\theta}}(x) \mid x, \boldsymbol{\theta})$. The product (or logarithmic summation) of the full dataset is then derived via Axiom 5. This is covered in more specifics in Implementation 1.

The normalisation element within the posterior, $P(D)$, ensures that the area under the curve of the posterior, such as in Figure 2, adds to 1. Hypothetically, it is possible to assign implicit distributions (without parameters) to the prior and likelihood (which would also include the model Ψ), and integrate $P(D)$ based on these implicit distributions.

Realistically, the evidence is intractable,

$$P(D) = \int_0^\infty p(D_y \mid D_x, \boldsymbol{\theta}_i) \cdot p(\boldsymbol{\theta}_i) d\boldsymbol{\theta}'$$

as it features the full model Ψ , and infinite possible values of $\boldsymbol{\theta}$ (made up of continuous variables). As a result, $P(D)$ is generally approximated, either via sampling or inference methods, sampling being more computationally intensive.

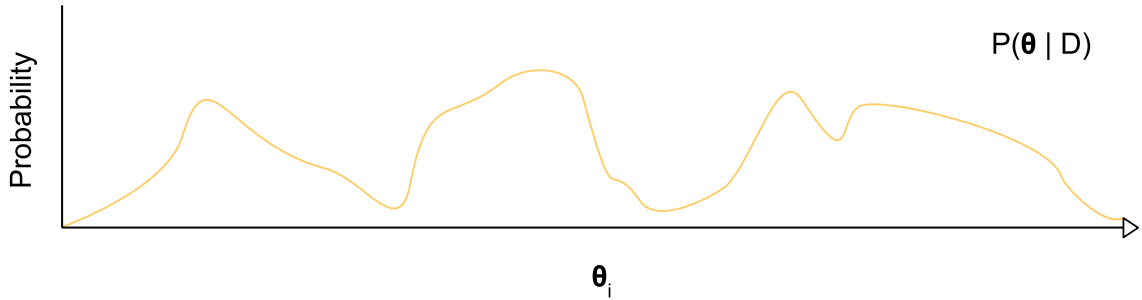


Figure 2: Sample graph to show how the probability of different values of $\boldsymbol{\theta}$ could differ in the instance of one specific training dataset D_0 and one specific neural network model Ψ . Note that $\boldsymbol{\theta}_i$ is multidimensional; this is a 1-dimensional representation for fit.

Proof 2 (The non-convex nature of a potential posterior.) *Consider the following context:*

- taking the quantity of neurons within the input and output layer of a neural network towards the limit of 0

- taking the quantity of points within a training dataset towards the limit of 0
- taking the quantity of possible values of θ within the network towards the limit of ∞
- taking the quantity of neurons within the hidden layers towards the limit of ∞
- a uniform prior associated with each possible θ
- calibrating the likelihood probabilities, $P(y \mid x, \theta)$, such that any model output need only be a comparatively small distance from the expected values of the training dataset for the parameter value θ to be considered a highly probable candidate, such that many values of θ are found within this range

As a result of the above, there would be a large quantity of values θ that would make plausible candidates as the parameters of the model, differing infinitely, in a continuous sense. Simultaneously, there are many different, equally valid ways to iterate through S_θ , from θ_i to θ_{i+1} , especially in the case of θ being multidimensional. As a result, the posterior is prone to fluctuate in a myriad of ways.

□

2.4.1 Hypothetical training method: brute-force

A simple, hypothetical approach would be to iterate over all the (uncountable) infinite possible values of θ , treating all with a (approximately zero, but non-zero probability, in the case that all elements of θ are continuous) uniform prior, and deducing the likelihood of each value of θ via the training dataset. As the evidence $P(D)$ would be the same for all values of $P(D \mid \theta)$, it is then just a matter of selecting the values of θ with the highest likelihood.

Typically brute-force is a viable method when there are only 1 or 2 unknown parameters. However, modern computational standards quickly begin to struggle in the realm of unknown multidimensional parameters of a higher degree.

2.4.2 Analytical methods

Hypothetically, it is possible to algebraically deduce the posterior. With very small scale neural networks, this is viable. However, at scale it quickly becomes intractable, as it involves a set of $|\theta|$ equations, which must then be solved simultaneously[SS06], for increasingly complicated models. Consider that modern LLMs feature neural networks where $|\theta| > 10^{12}$.

2.4.3 Sampling-based training method: MCMC Metropolis

The MCMC algorithm works by approximating a function that samples can be drawn from, $P(\theta \mid D)$, using a function that is proportional to it, $P(D \mid \theta)P(\theta)$, the unnormalised posterior, which is an unscaled version of $P(\theta \mid D)$. Values of θ are drawn in an iterative, potentially

stochastic process, and unnormalised posteriors (which all have the same normalisation factor of $\frac{1}{P(D)}$) compared between a previously generated θ and a newly proposed value θ_P . A probability based on the proportion between the unnormalised posteriors of θ and θ_P is then compared to a uniform random variable, and if it performs better than the random variable (i.e. better than randomly sampling θ), it is accepted into the chain. The process loops to gradually build a chain which can climb to a local maximum; note that ‘warm restarts’, whereby a new value θ is generated at random, are required to explore the full posterior. The method involves many hyperparameters such as step size of θ , quantity of samples of θ to take, distributions to use when generating θ_P from θ . It can be very computationally intensive, and is difficult to scale due to the inability to match computationally efficient hyperparameters to the unknown posterior distribution.

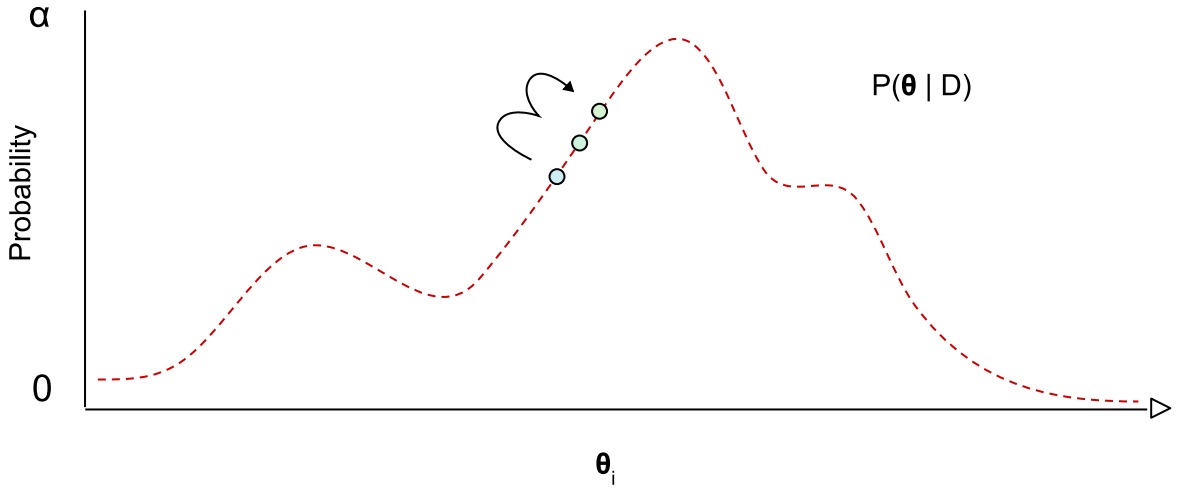


Figure 3: sample graph to show approximating an unnormalised posterior $P(\theta | D)$ being approximated via MCMC; $\alpha \in (0, \infty)$. MCMC gradually iterates around the posterior after a random initialisation, and will become stuck once reaching a local maximum. The exact posterior is unknown, hence the dotted line. Note that θ_i is multidimensional; this is a 1-dimensional representation for fit.

Chandra & Simmons, 2016 [CS24] demonstrated an implementation to approximate a posterior via MCMC sampling, without 3rd-party library dependencies (they had not yet become commonplace). This is detailed in Implementation 1 (omits Langevin gradient descent). This same algorithm is also seen in Hands-on primer BNNs Algorithm 4 [JLB⁺20], in the symmetric case.

Implementation 1 (MCMC) *By Chandra & Simmons*

Part 1: declaration of distributions and parameters, and their representations

1. A model Ψ is defined which takes model parameters θ and training dataset values x , plus

noise, the noise having a variance of τ^2

$$\hat{y} = \Psi_{\boldsymbol{\theta}}(x) + e$$

$$e \sim \mathcal{N}(0, \tau^2)$$

2. The likelihood of each model output is defined to follow a normal distribution

$$P(y \mid x, \boldsymbol{\theta}, \tau^2) \sim \mathcal{N}(\Psi_{\boldsymbol{\theta}}(x), \tau^2)$$

3. By assuming all pairs of events (x, y) within the dataset are independent, via Axiom 5 the product of them will be equivalent to the probability of the whole dataset D occurring, given the prior distribution [JLB⁺20];

$$P(D_y \mid D_x, \boldsymbol{\theta}) = \prod_{(x,y) \in D} P(y \mid x, \boldsymbol{\theta}, \tau^2)$$

4. The likelihood is then represented in log format to prevent the product of probabilities tending towards 0

$$\ln\{P(D_y \mid D_x, \boldsymbol{\theta})\} = \sum_{(x,y) \in D} \ln\{P(y \mid x, \boldsymbol{\theta}, \tau^2)\}$$

5. The probability of the log-likelihood is then represented via an algebraic form of the multivariate Gaussian distribution

6. Prior beliefs about the underlying distribution that elements of $\boldsymbol{\theta}$ and τ follow are formed

$$\boldsymbol{\theta} \sim \mathcal{N}(0, \Sigma)$$

$$\tau \sim IG(v_1, v_2)$$

7. The probability of the log prior is then represented via an algebraic form of the multivariate Gaussian distribution

Part 2: declaration of sampling loop variables

- Quantity of MCMC samples, plus burn-in quantity
- Set learning rate step sizes for $\boldsymbol{\theta}$, τ ; $s_{\boldsymbol{\theta}}$, s_{τ}
- Hyperparameter Σ (multivariate standard deviation)
- Initial numerical value for likelihood probability p_L , for later comparison
- Architecture of neural network Ψ
- Dimension of $\boldsymbol{\theta}$ is derived via architecture of neural network Ψ

- *Initial value for θ based on random values*

Part 3: main sampling loop; Metropolis algorithm

1. *Propose a new value of θ_p , $\theta_p \sim \mathcal{N}(\theta, s_\theta)$*
2. *Generate a proposal for a new prior and likelihood using θ_p and Σ*
3. *Find the differences between the likelihood and the proposed likelihood, and the same for the prior and proposed prior (not a fraction due to logarithms)*
4. *Add the logarithmic differences (not multiply as in Bayes rule, due to logarithms), and run through $\exp()$ to convert back from logarithms <input probability based on two unnormalised posteriors with different params here>*
5. *Draw a random probability from a uniform distribution $U(0, 1)$*
6. *If the probability of the new proposal is better than a random probability (with an average of 0.5), accept the new proposal, and adjust variables accordingly*

$$\theta = \theta_p$$

7. *Loop back to start until quantities of burn-in samples and MCMC samples complete*
-

Note that the above implementation features the multiplication of probabilities of all elements of θ , to generate a numerical value for the prior, and this is done directly after sampling from a distribution, but also after combining the samples with the result of a previous training iteration. Probability theory requires that elements of θ are independent, as in Axiom 5, for the product rule to apply. Neural network parameters are generally not considered to be entirely independent of one another, after training; specifically, manually changing a subset of θ in a significant way, during training, may result in a different set of resulting parameters θ , post-training. This may be a trade-off due to limited compute availability, a standard issue with Bayesian neural networks.

2.4.4 Inferential training method: Variational Inference

VI takes the approach of approximating the posterior via a simpler distribution, $q_\xi(\theta)$, where ξ is the parameters of the distribution. It then becomes a matter of deducing the most fitting distribution parameters, such as in Figure 4.

The degree of similarity between $q_\xi(\theta)$ and $P(\theta \mid D)$ is measured via KL-divergence, a statistical measurement developed at the NSA in the 1950s [KL51].

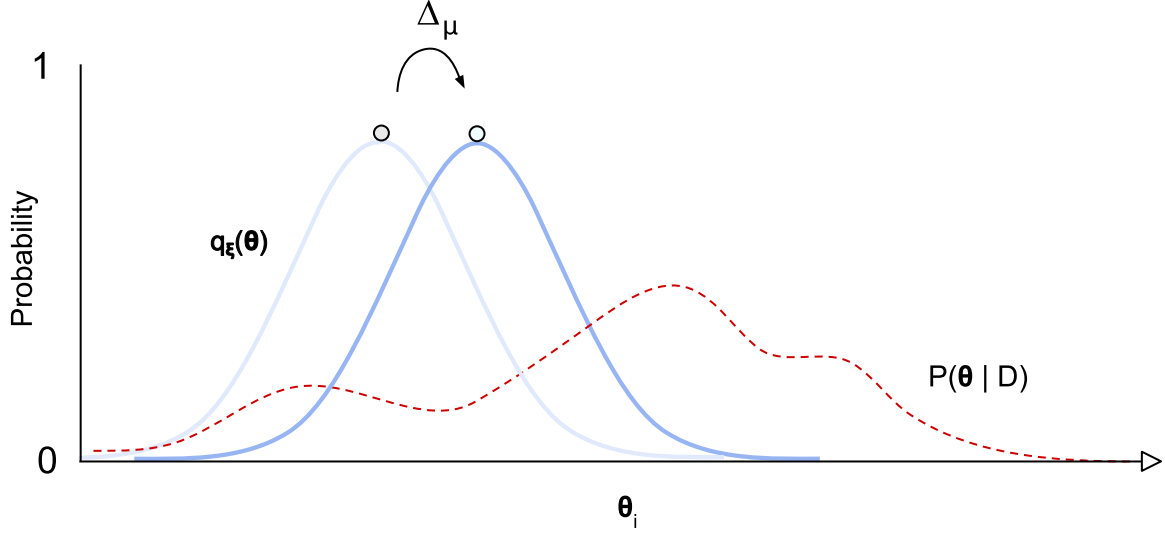


Figure 4: sample graph to show approximating a normalised posterior $P(\boldsymbol{\theta} \mid D)$ via a Gaussian distribution, $q_{\xi}(\boldsymbol{\theta})$. Consider that the area under the curve of the normal distribution must add up to 1, and the same for the normalised posterior, and therefore that the modes of both distributions may not match up. Hence, the imperfections of approximating a multimodal distribution with a common probability distribution. The exact posterior is unknown, hence the dotted line. Note that $\boldsymbol{\theta}_i$ is multidimensional; this is a 1-dimensional representation for fit.

Theorem 2 (KL Divergence)

$$\text{KL}(q_{\xi}(\boldsymbol{\theta}) \parallel p(\boldsymbol{\theta} \mid D)) = \int q_{\xi}(\boldsymbol{\theta}) \log\left(\frac{q_{\xi}(\boldsymbol{\theta})}{p(\boldsymbol{\theta} \mid D)}\right) d\boldsymbol{\theta} \quad (2)$$

Proof 3 *The validity of KL-Divergence as a loss function for Bayesian Neural Networks.*

The logarithmic division can be rearranged such that the logarithm of one is subtracted from the logarithm of the other.

This is a straightforward difference between two distributions. However, this subtraction alone does not account for the priority in the context of Bayesian Neural Networks, which is to find the most probable values of $\boldsymbol{\theta}$; the maximum a posteriori.

Multiplying the logarithmic difference by the approximating distribution $q_{\xi}(\boldsymbol{\theta})$ when it is at its maximum will cause the difference between the distributions $p(\boldsymbol{\theta} \mid D)$ and $q_{\xi}(\boldsymbol{\theta})$ at this point to have the largest effect on the KL-Divergence score. This approximation peak is the approximation of the maximum a posteriori, where the most sought after values of $\boldsymbol{\theta}$ are found. The same is true for other large values of $q_{\xi}(\boldsymbol{\theta})$.

□

The KL-divergence formula can then be expanded as follows:

$$\begin{aligned}
\text{KL}(q_{\xi}(\boldsymbol{\theta}) \parallel p(\boldsymbol{\theta} \mid D)) &= \int_{\boldsymbol{\theta}} q_{\xi}(\boldsymbol{\theta}) \log \left(\frac{q_{\xi}(\boldsymbol{\theta})}{P(\boldsymbol{\theta} \mid D)} \right) d\boldsymbol{\theta} \\
&= \int_{\boldsymbol{\theta}} q_{\xi}(\boldsymbol{\theta}) \log \left(\frac{q_{\xi}(\boldsymbol{\theta}) \cdot P(D)}{P(D \mid \boldsymbol{\theta}) \cdot P(\boldsymbol{\theta})} \right) d\boldsymbol{\theta} \\
&= \int_{\boldsymbol{\theta}} q_{\xi}(\boldsymbol{\theta}) \left[\log \left(\frac{q_{\xi}(\boldsymbol{\theta})}{P(\boldsymbol{\theta})} \right) - \log P(D \mid \boldsymbol{\theta}) + \log P(D) \right] d\boldsymbol{\theta} \\
&= \int_{\boldsymbol{\theta}} q_{\xi}(\boldsymbol{\theta}) \log \left(\frac{q_{\xi}(\boldsymbol{\theta})}{P(\boldsymbol{\theta})} \right) d\boldsymbol{\theta} - \int_{\boldsymbol{\theta}} q_{\xi}(\boldsymbol{\theta}) \log P(D \mid \boldsymbol{\theta}) d\boldsymbol{\theta} + \int_{\boldsymbol{\theta}} q_{\xi}(\boldsymbol{\theta}) \log P(D) d\boldsymbol{\theta} \\
&= \text{KL}(q_{\xi}(\boldsymbol{\theta}) \parallel P(\boldsymbol{\theta})) - \mathbb{E}_{q_{\xi}}[\log P(D \mid \boldsymbol{\theta})] + \log P(D)
\end{aligned} \tag{3}$$

Regarding the last step:

- As a result of Axiom 2, the integrand of a probability distribution function is 1, and $\log P(D)$ can be considered a constant when integrating in respect to $\boldsymbol{\theta}$
- Thinking back to the fundamentals of integration, the multiplication of a series of (infinite, or scaled down for mental comprehension) rectangles between q_{ξ} (e.g. q_{ξ} being a Gaussian distribution) and $P(D \mid \boldsymbol{\theta})$ will be consistent, provided the parameters associated of the probability distribution, ξ , do not change

The following observations can be made:

- $\text{KL}(q_{\xi}(\boldsymbol{\theta}) \parallel p(\boldsymbol{\theta} \mid D))$ is an optimisation problem, whereby the objective is minimisation
- The integrands are over the full set $S_{\boldsymbol{\theta}}$ and are therefore intractable to integrate numerically
- $\text{KL}(q_{\xi}(\boldsymbol{\theta}) \parallel P(\boldsymbol{\theta})) \approx 0$ when the two distributions are a perfect match, and tends towards 1 when they are a bad match
- $\mathbb{E}_{q_{\xi}}[\log P(D \mid \boldsymbol{\theta})]$ contains the likelihood, which contains the full neural network Ψ , which is intractable to integrate analytically
- $\log P(D \mid \boldsymbol{\theta})$ will be 0 when a value of $\boldsymbol{\theta}$ is a perfect match for the data
- The dataset during training a neural network typically remains static, therefore $\log(P(D))$ is a constant during training, unaffected by values of $\boldsymbol{\theta}$

The following deductions can then be made:

- Minimising the terms $\text{KL}(q_{\xi}(\boldsymbol{\theta}) \parallel P(\boldsymbol{\theta})) - \mathbb{E}_{q_{\xi}}[\log P(D \mid \boldsymbol{\theta})]$ such that $\text{KL}(q_{\xi}(\boldsymbol{\theta}) \parallel p(\boldsymbol{\theta} \mid D))$ tends towards zero will give an approximation of the value of $\log(P(D))$
- The parameters of the distribution of the prior, $P(\boldsymbol{\theta})$, which generates a value of $\boldsymbol{\theta}$, are a focus of the optimisation problem

- As the full posterior distribution $P(\boldsymbol{\theta} \mid D)$ cannot be easily derived, the parameters $\boldsymbol{\xi}$ of $q_{\boldsymbol{\xi}}(\boldsymbol{\theta})$ are also a focus of the optimisation problem

More commonly, the optimisation function described in the deductions above is known as the ELBO, and is a negated form of the KL-divergence expansion above, and as a result the objective is instead for maximisation.

Theorem 3 (ELBO: Evidence lower-bound)

$$L(\boldsymbol{\theta}, \boldsymbol{\xi}) = \mathbb{E}_{q_{\boldsymbol{\xi}}}[\log P(D|\boldsymbol{\theta})] - \text{KL}(q_{\boldsymbol{\xi}}(\boldsymbol{\theta}) \parallel P(\boldsymbol{\theta})) \quad (4)$$

Consider Proof 1. Instead of associating separate distributions for $P(\boldsymbol{\theta})$ and $q_{\boldsymbol{\xi}}(\boldsymbol{\theta})$, and comparing them via the KL-divergence, which would result in a complexity of $O(n^{|\boldsymbol{\alpha}|+|\boldsymbol{\xi}|})$ parameter permutations, to generate a value $\boldsymbol{\theta}$, it is instead possible to consider only $O(n^{|\boldsymbol{\xi}|})$ parameters by basing $P(\boldsymbol{\theta})$ upon $q_{\boldsymbol{\xi}}(\boldsymbol{\theta})$, where n represents the quantity of parameters within the neural network.

Proof 4 (The validity of basing $P(\boldsymbol{\theta})$ upon the $q_{\boldsymbol{\xi}}(\boldsymbol{\theta})$ distribution) *Recall that the ELBO contains the term $\text{KL}(q_{\boldsymbol{\xi}}(\boldsymbol{\theta}) \parallel P(\boldsymbol{\theta}))$, which means finding two distributions, one for $P(\boldsymbol{\theta})$ and one for $q_{\boldsymbol{\xi}}(\boldsymbol{\theta})$, which are as similar as possible.*

Consider the value $\boldsymbol{\theta}_a = \arg \max_{\boldsymbol{\theta} \in S_{\boldsymbol{\theta}}} P(D|\boldsymbol{\theta})$

If $\boldsymbol{\theta}_a$ is drawn from a probability distribution associated with $P(\boldsymbol{\theta})$, then, if an equivalent probability distribution is associated with $q_{\boldsymbol{\xi}}(\boldsymbol{\theta})$, it should also be possible to maximise $q_{\boldsymbol{\xi}}(\boldsymbol{\theta})$ via the same value $\boldsymbol{\theta}_a$, when $\boldsymbol{\theta}_a$ is sampled from $q_{\boldsymbol{\xi}}(\boldsymbol{\theta})$ and the parameters $\boldsymbol{\xi}$ are calibrated accordingly.

Furthermore, as $\text{KL}(q_{\boldsymbol{\xi}}(\boldsymbol{\theta}) \parallel P(\boldsymbol{\theta}))$ optimises when two distributions are as similar as possible, a simple model is to make them the same distribution, and based on the same parameters.

Therefore, $q_{\boldsymbol{\xi}}(\boldsymbol{\theta})$ can initially be based upon $P(\boldsymbol{\theta})$, which (unless an extremely accurate prior) will likely lead to $\log P(D|\boldsymbol{\theta})$ indicating inaccurate values of $\boldsymbol{\xi}$, and then the parameters of $q_{\boldsymbol{\xi}}(\boldsymbol{\theta})$ iterated from there, and then $P(\boldsymbol{\theta})$ adjusted with each update (somewhat similar to Bayesian updating).

□

Theorem 4 (Reduced ELBO)

$$R(\boldsymbol{\theta}, \boldsymbol{\xi}) = \mathbb{E}_{q_{\boldsymbol{\xi}}}[\log P(D|\boldsymbol{\theta})] = \int_{\boldsymbol{\theta}} q_{\boldsymbol{\xi}}(\boldsymbol{\theta}) \log P(D|\boldsymbol{\theta}) d\boldsymbol{\theta} \quad (5)$$

Inspecting next what $\int_{\boldsymbol{\theta}} q_{\boldsymbol{\xi}}(\boldsymbol{\theta}) \log P(D|\boldsymbol{\theta}) d\boldsymbol{\theta}$ represents. In the case that $P(D|\boldsymbol{\theta})$ featured the simplest possible linear regression model, then $|\boldsymbol{\theta}| = 1$, and therefore if $q_{\boldsymbol{\xi}}(\boldsymbol{\theta})$ were represented by infinitely small rectangles, $\mathbb{E}_{q_{\boldsymbol{\xi}}}[\log P(D|\boldsymbol{\theta})]$ would mean the expected value of $P(D|\boldsymbol{\theta})$ based on the probability of different values of $\boldsymbol{\theta}$, given that $\boldsymbol{\xi}$ is static whilst considering the different values of $\boldsymbol{\theta}$. Therefore, different values of $\boldsymbol{\xi}$ will lead to different values of $\mathbb{E}_{q_{\boldsymbol{\xi}}}[\log P(D|\boldsymbol{\theta})]$. As considering all possible values of $\boldsymbol{\theta}$ is a computational bottleneck, probability distributions are instead assigned to each possible value of θ_i , $\theta_i \in \boldsymbol{\theta}$. This reduces the range of $\boldsymbol{\theta}$, but a similar bottleneck remains; all permutations of $\boldsymbol{\theta}$ cannot be generated. However, the ability to draw samples from the associated distributions becomes an option, as a model, the parameters of the probability distributions can then be iterated in the context of a sufficient quantity of samples.

In non-Bayesian neural networks, the weights of the network, $\boldsymbol{\theta}$, are adjusted in minor ways with the intention of minimising/maximising the optimisation function, via numerical partial derivatives and gradient descent. However, in the context of Bayesian neural networks, because each element θ_i , $\theta_i \in \boldsymbol{\theta}$, is drawn from a probability distribution, this means that taking the partial derivative of the optimisation function, L , in respect to an instance of θ_i , $\frac{\partial L}{\partial \theta_i}$, is not feasible, as it is drawn with probabilistic stochasticity, which is not deterministic.

Following on from Proof 4, instead, a method known as reparameterisation is used, as shown in Figure 5. This means using a deterministic intermediate function, t , which treats the probability distribution $q_{\boldsymbol{\xi}}$ parameters $\boldsymbol{\xi}_i$ of the node i within the neural network, and a value generated via an equivalent distribution, $q(\epsilon)$, as constants.

The above reasoning leads to a method known as Bayes-by-backpropagation, as typical neural network backpropagation (as found in non-Bayesian neural networks) becomes viable, but whereby $\boldsymbol{\xi}$ is instead iteratively adjusted, as opposed to $\boldsymbol{\theta}$ directly. [BCKW15] [GF20]

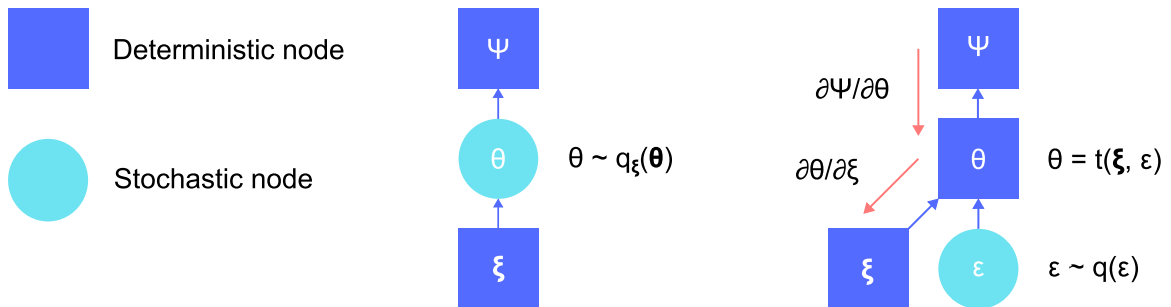


Figure 5: diagram to show how a stochastic node can be remapped to a deterministic mode via an intermediate function, which therefore permits a partial derivative to be taken, and therefore backpropagation. Inspired and adapted from MIT 6.S191 slide on variational autoencoders.

The following derivation is then adapted from the original Bayes-by-backpropagation paper [BCKW15].

$$f(\boldsymbol{\theta}, \boldsymbol{\xi}_a) = \log \frac{q_{\boldsymbol{\xi}_a}(\boldsymbol{\theta})}{P(\boldsymbol{\theta})} - \log P(D \mid \boldsymbol{\theta})$$

$$\begin{aligned} \frac{\partial}{\partial \boldsymbol{\xi}_a} \mathbb{E}_{q_{\boldsymbol{\xi}_a}}[f(\boldsymbol{\theta}, \boldsymbol{\xi}_a)] &= \frac{\partial}{\partial \boldsymbol{\xi}_a} \int_{\boldsymbol{\theta}} q_{\boldsymbol{\xi}_a}(\boldsymbol{\theta}) \cdot f(\boldsymbol{\theta}, \boldsymbol{\xi}_a) d\boldsymbol{\theta} \\ &= \frac{\partial}{\partial \boldsymbol{\xi}_a} \int_{\epsilon} p(\epsilon) \cdot f(\boldsymbol{\theta}, \boldsymbol{\xi}_a) d\epsilon \\ &= \mathbb{E}_{q_{\epsilon}} \left[\frac{\partial f(\boldsymbol{\theta}, \boldsymbol{\xi}_a)}{\partial \boldsymbol{\theta}} \frac{\partial \boldsymbol{\theta}}{\partial \boldsymbol{\xi}_a} + \frac{\partial f(\boldsymbol{\theta}, \boldsymbol{\xi}_a)}{\partial \boldsymbol{\xi}_a} \right] \end{aligned}$$

Where $\boldsymbol{\xi}_a$, represents a matrix made up of one element of the set of parameters of the PDF associated with each instance of θ_i , $\theta_i \in \boldsymbol{\theta}$, $\boldsymbol{\xi}_a \in \boldsymbol{\xi}_i$.

The following constraints then apply:

- As each value of θ_i is generated, $\theta_i \in \boldsymbol{\theta}$, each instance of $\boldsymbol{\theta}$ will have a set of associated PDF parameters $\boldsymbol{\xi}_i$, differentiation will then need to be performed $|\boldsymbol{\xi}_i| \cdot |\boldsymbol{\theta}|$ times (once per parameter)
- As $q(\epsilon)$ is of the same probability density function as $q_{\boldsymbol{\xi}}(\boldsymbol{\theta})$, but with fixed parameters during training and inference, the parameters of $q(\epsilon)$ are treated as hyperparameters (in the case that q is of Gaussian distribution, commonly a tuple of $(0, 1)$ fulfills (μ, σ) , due to the similarity with non-Bayesian neural network weights, and the issues of exploding/vanishing gradients that can occur with backpropagation)

Note that, in the above derivation, the parameters are then incrementally changed using a given step size, α . An application of this can be seen in the original Bayes-by-backpropagation paper, [BCKW15] for which an example of the function t , could be, in the case of a Gaussian distribution being associated with $q_{\boldsymbol{\xi}}$:

$$\boldsymbol{\xi} = (\mu, \rho)$$

$$t(\boldsymbol{\xi}, \epsilon) = \mu + \log(1 + \exp(\rho)) \circ \epsilon$$

Note the similarity of t to drawing from a Gaussian distribution.

The eventual outcome of training should be that, as each θ_i within the Bayesian neural network has a parameter set $\boldsymbol{\xi}_i$ associated with it, a variety of values of $P(\boldsymbol{\theta} \mid D)$ can be found, whereby these values of $P(\boldsymbol{\theta} \mid D)$ are close to the maximum a posteriori; that is to say, a variety of values of $\boldsymbol{\theta}$ that fit the data D .

2.5 Utilising Bayesian neural networks

Once the posterior has been derived (the most probable values of $\boldsymbol{\theta}$ that generate closely the outputs found in the training dataset, when the corresponding inputs from the training dataset are input into $\Psi_{\boldsymbol{\theta}}$), this can be used to build probability distributions when the model is provided

with data not found in the training dataset.

Consider a prediction wanted to be made using Ψ_{θ_i} and a new input x_a . x_a is run through the model with an instance of θ_i , $\Psi_{\theta_i}(x_a)$, and then the output of the model converted to a probability, such as the binary classifier featured in Example 3. This prediction probability should then be multiplied by the probability that the value θ_i itself is accurate (i.e. it correctly fits the underlying distribution that the data is being drawn from), to gauge the accuracy of the prediction. This is to be repeated for a wide range of values of θ_i , so that confidence intervals can be derived.

$$P(\Psi(x), \theta_1 | D) = P(\Psi(x) | \theta_1) \cdot P(\theta_1 | D)$$

$$\therefore \mathbb{E}[p(\Psi(x), \theta | D)] = \int_{\theta} P(\Psi(x) | \theta) \cdot P(\theta | D) d\theta$$

As x and D are constants in the above, the graph that the above integrand represents would be of a probability distribution for the probability of the output $\hat{y}$ (based on the probability of different values of θ), against the set of values T_{θ} selected, $T_{\theta} \subseteq S_{\theta}$, used to make the predictions; as in Figure 6. The probabilities generated can then be grouped based on intervals of θ_i , to give confidence intervals. For example, θ_a may fit a dataset D_a very well in the context of a specific model which performs binary classification, such that when x_b is input into the model and assigned a probability:

$$\hat{y}_b = \Psi_{\theta_a}(x_b)$$

$$P(\hat{y}_b | \theta_a) > 0.5$$

$$P(\hat{y}_b | \theta_a) \cdot P(\theta_a | D) \approx \max_{\theta \in S_{\theta}} P(\hat{y}_b | \theta) \cdot P(\theta | D)$$

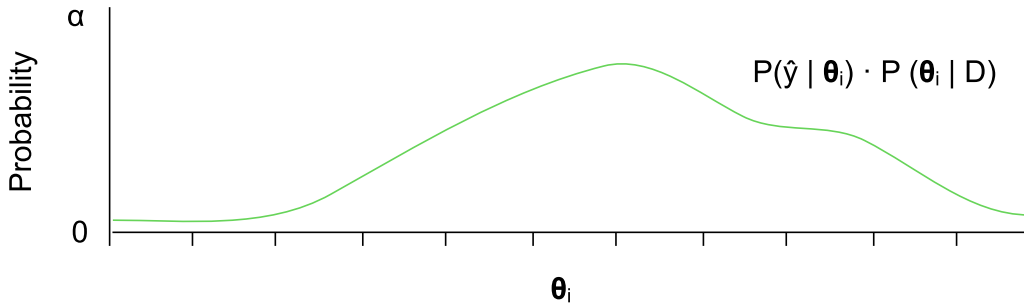


Figure 6: sample graph, inspired by the Peak District, of different probabilities of one model output, $\hat{y}_1 = \Psi_{\theta_1}(x_1)$, derived via probabilities of different values of θ_i , where $\alpha \in (0, 1]$. Note that θ_i is multidimensional; this is a 1-dimensional representation force fit.

More formally, a quantity N of samples θ_i are made from the posterior, for which $P(\theta_i | D) > c$ for some chosen constant c , and a new input x is provided to N instances of model Ψ_{θ_i} , such that each model generates an output $\hat{y}$ based on x and θ_i , $\hat{y} = \Psi_{\theta_i}(x)$. This process will produce the set of tuples $(\hat{y}, \theta)$, $\hat{y} \in S_{\hat{y}}$, $\theta \in S_{\theta}$. Every instance of $\hat{y}$ is then converted to a probability

via an editorially selected probability distribution with constant parameters, Q_{α} (modeled on D_y), $p_{\hat{y}} = Q_{\alpha}(\hat{y})$, which equivalently represents $P(\hat{y} \mid \theta)$, such that the aforementioned tuple can be extended to $(p_{\hat{y}}, \hat{y}, \theta)$, each instance of which can be multiplied by its associated value of $P(\theta \mid D)$.

$$\begin{aligned} p_{\hat{y}} &= Q_{\alpha}(\hat{y}) \\ &= P(\hat{y} \mid \theta_i) \\ &= P(\Psi_{\theta_i}(x) \mid \theta_i) \quad \theta_i \in S_{\theta} \end{aligned}$$

2.5.1 Limitations of separate training & utilisation stages

Equivalently to non-Bayesian neural networks, the training stage is still generally distinct from the utilisation stage, and this means that Bayesian neural networks, whilst being able to better account for unknowns in the underlying population (via the training data at training time), may still experience the same issues of overfitting to the training data, if the underlying population gradually changes over time.

For example, in the case of predicting travel times on public transport, a set of Bayesian neural networks may contribute to more thorough predictions based on passenger data recently collected, but may gradually become inaccurate over time as changes in a local economy cause passengers to use the services in a different way.

2.6 Credal Bayesian Deep Learning

2.6.1 Overview of CBDL

Typically, when estimating only 1 parameter, such as a biased coin, a relatively accessible quantity of data points may be all that is required [SS06] to derive the underlying distribution, regardless of initial assumptions. Extrapolating the idea to neural networks: essentially, with more data, the influence of the prior becomes reduced, as $P(D \mid \theta)$ increases in accuracy. As a result, values of θ can be derived via the posterior, $P(\theta \mid D)$, that fit the underlying distribution of the data and the architecture, such that both training data and new data points that occur during utilisation, can be consistently correlated with accurate predictions.

However, as $|\theta|$ increases, as is typical with highly-scaled modern neural networks, it is possible that the ratio of $|\theta|$ to $|D|$ means that it is not possible to consistently produce the same posterior, as the prior distribution or likelihood selected is varied (recall that the likelihood, $P(D \mid \theta)$, can also be depicted as $P(\Psi_{\theta}(D_x) \mid D_x, \theta)$, and thereby features the full model architecture). There exists ambiguity in the ideal probability model of the underlying data distribution.

Formally this ambiguity is described as aleatoric uncertainty and epistemic uncertainty.

Aleatoric uncertainty (AU) The probabilistic stochasticity of a random variable, defined in terms of a probability distribution and associated parameters. Specifically, once the parameters of the distribution a variable follows are known, the extent to which a sample from the distribution may vary.

Epistemic uncertainty (EU) The uncertainty of the distribution and distribution parameters of a variable that follows an unknown probability distribution; with more data, the uncertainty of the distribution and associated parameters can be reduced.

One approach is to aim to reduce epistemic uncertainty; for example to consider smaller neural networks that are expected to fit the limitations of the available data D_α better, such that for all values of Ψ_i and θ_i which exist within these limitations:

$$|\theta_i| \propto |D_\alpha|$$

$$\Psi_i \wedge |\theta_i| \wedge |D_\alpha| \rightarrow EU(P_{posterior}) \approx 0$$

This comes potentially at the cost of the capability of the model to identify more complex patterns in data available (e.g. to make predictions to greater precision), akin to an animal with a brain with fewer neurons than a human being only capable of simpler predictions. In the realm of financial analysis, for example, this may come at competitive cost, or in the realm of medicine, at the cost of an earlier detection.

An alternate approach is to tolerate and measure epistemic uncertainty, by continuing to scale the neural network, increasing $|\theta|$ to increase the level of precision of predictions, and then presenting better the uncertainty of predictions made. If varying the prior distributions $P(\theta)$, or the model via $P(D | \theta)$, has a significant effect on the posterior, it can be ascertained that there exists significant epistemic uncertainty.

CBDL aims to address model uncertainties by considering a variety of possible prior distributions of θ and model architectures. CBDL involves specifying the extremes of a set of target priors, specified exP_{prior} , and the extremes of the set of target architectures, specified in $exP_{likelihood}$. An FGCS is then generated for each, resulting in P_{prior} and $P_{likelihood}$, where:

$$exP_{prior} \subseteq P_{prior}$$

$$exP_{likelihood} \subseteq P_{likelihood}$$

$$|exP_{prior}| \geq 1 \quad |exP_{likelihood}| \geq 1$$

An example of this generation is formally defined in the original CBDL paper, in the case of combining Gaussian distributions [CDJ⁺23].

All possible combinations of (l, p) , where $l \in P_{likelihood}$ and $p \in P_{prior}$, are then combined and trained (such as via VI), to generate $P_{posterior}$, where $P_{posterior}$ is made up of a set of distributions

of $P(\boldsymbol{\theta}|D)$, each posterior distribution featuring the most probable values of $\boldsymbol{\theta}$ for one instance of an associated Bayesian neural network. Each instance may be represented as:

$$(l_i, p_j, P(\boldsymbol{\theta}|D)_{(i,j)})$$

$$i \in \mathbb{N}, j \in \mathbb{N}$$

After training, during utilisation, a set $P_{\text{predictive}}$ is generated, such that one distribution of probabilities, $P(\hat{y} | \boldsymbol{\theta})$, for a new input x , are generated for each posterior $P(\boldsymbol{\theta}|D)$; one-for-one mapping between $P(\boldsymbol{\theta}|D)$ and $P(\hat{y} | \boldsymbol{\theta})$. Epistemic and aleatoric uncertainties can then be derived via the sets.

Ultimately, CBDL aims to measure predictive inaccuracies which may result from the specification of only one prior and likelihood (a more typical approach to Bayesian neural networks), each of which are essentially hyperparameters.

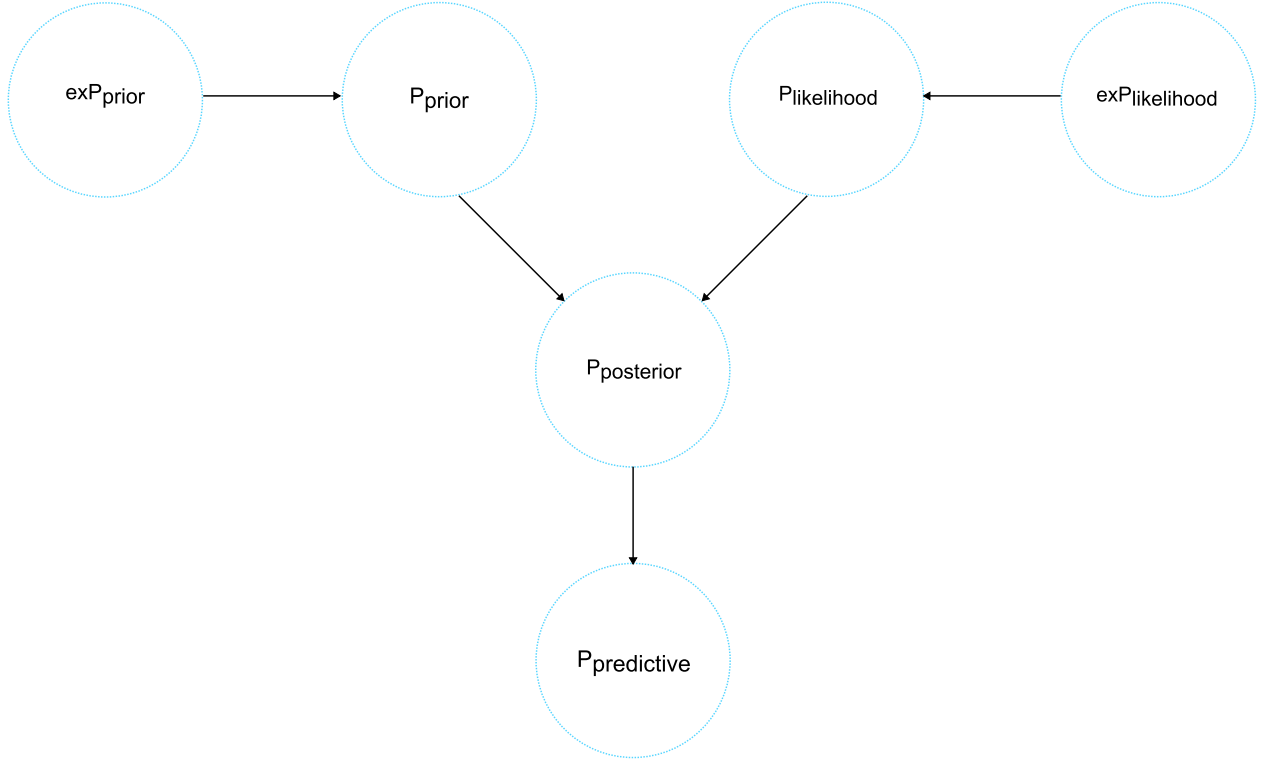


Figure 7: diagram to show how CBDL sets are derived from one another.

2.6.2 Prerequisite knowledge for CBDL

This subsection is an alternate presentation of the prerequisite knowledge featured in the original CBDL paper, and uses the same syntax and notation as defined within it.

Π is a set of probability distributions P_i , $i \in \mathbb{N}$, all applicable to the same sample space Ω , which are each unique such that any P_i cannot be generated by a combination of the other probability distributions within Π .

Π' is generated by combining the elements of Π in every possible combination, under the condition that the combined proportions of the distributions (used to generate a new distribution) is equal to 1 for each distribution generated.

Meaning of lower & upper probabilities $\underline{P}(A)$, $\overline{P}(A)$

Essentially, upper and lower probabilities regard considering:

- a set of probability distributions, Π'
- a set of interval points or discrete events, A
- all the possible integrands/summations based upon the above elements (one for each probability distribution)

...and then selecting the lowest probability output of the integrands/summations within Π' , then deducing the upper probability from there.

In the case that A is made up of interval positions within Ω , for example (α, β) , and Π' represents a set of continuous probability distributions, take the interval positions within A , and find the minimum possible integrand based on the elements of Π' and A :

$$\underline{P}(A) = \min_{P \in \Pi'} \int_{\alpha}^{\beta} P(\omega) d\omega$$

In the case that Π' contains discrete probability distributions, take the subset of Ω , A , and find the minimum possible summation based on the elements within A :

$$\underline{P}(A) = \min_{P \in \Pi'} \sum_{i=0}^{N \leq |A|} P(\omega_i)$$

$\overline{P}(A)$ is then defined as $1 - \underline{P}(A)$.

There is a relation between Π and Π' regarding lower/upper probabilities: a probability value can only be considered lower/upper for Π if it is also an lower/upper probability for Π' .

Confidence regions

$$R_{\alpha} : \Pi \times \Omega \rightarrow \mathcal{P}(\Omega)$$

R_{α} takes a convex hull of probability distributions and the sample space, and outputs a subset of the sample space, for which the integrand/summation, in respect to all elements of Π' , is greater than $(1 - \alpha)$. For example $(\{P_1, P_2, \dots, P_k\}, \{\omega_1, \omega_2, \omega_3\})$ as input, set $\{\omega_1, \omega_2\}$ as output.

A set $S_{\omega} = R_{\alpha}(\Pi', \Omega)$ is categorised as Imprecise Highest Density Region if:

1. The lower probability $\underline{P}(A)$, i.e. the smallest integrand, or smallest summation, among all P_i applied to S_ω (where $P_i \in \Pi'$), is greater than or equals to $(1 - \alpha)$.
2. $|S_\omega|$ is as small as possible, compared to all other possible values of $R_\alpha(\Pi', \Omega)$.

$$IR_\alpha(\Pi', \Omega) \subset R_\alpha(\Pi', \Omega)$$

$IR_\alpha(\Pi', \Omega)$ is an instance of IHDR.

A HDR is defined equivalently to an IHDR, except in terms of one probability distribution only; that is, the integrand/summation is to be greater than $(1 - \alpha)$, and $|S_\omega|$ to be as small as possible. ‘Highest density’ in HDR means where in the probability distribution (either intervals or discrete events) the probabilities are largest, in terms of integrating or summing to greater than $(1 - \alpha)$. HDRs are the Bayesian equivalent to means of applying confidence intervals (akin to classical statistics).

Aleatoric & epistemic uncertainty

The Shannon entropy function, H , indicates the shape of a distribution; whereby the output increases as uniformity of the distribution increases.

$$H : \mathbb{R}^n \rightarrow \mathbb{R}$$

The Shannon entropy can be applied to the set Π' to generate values known as the upper entropy $\overline{H}$ and lower entropy $\underline{H}$, by calculating the Shannon entropy for all probability distributions P_i , $P_i \in \Pi'$, and selecting the highest and lowest outputs of H .

Aleatoric Uncertainty is measured as the lower entropy, Epistemic Uncertainty is measured as the difference between the upper entropy and the lower entropy:

$$\overline{H}(P') = \underline{H}(P') + [\overline{H}(P') - \underline{H}(P')]$$

A large difference between the upper entropy and lower entropy means that the distributions’ level of uniformity within Π' varies widely, meaning, in the context of Bayesian neural networks, the distributions that best fit the data are unknown.

3 Experiments

All programming relevant to experiments is open-source and available at:

DOI [10.5281/zenodo.19661697](https://doi.org/10.5281/zenodo.19661697)

3.1 Experiment #1: Dirichlet distribution

3.1.1 Purpose

This experiment explores CBDL methods outside of a neural network context, to give a simple overview of the general idea.

3.1.2 Methodology

The following experiment will be oriented more around inspecting CBDL methods, as opposed to the complexities of approximating the posterior (computationally expensive, compatibility issues with major ML library implementations), and therefore will utilise an analytical method for training, via a Bayesian concept known as a conjugate prior, that will circumvent the integration bottleneck.

A conjugate prior works on the premise that, when the algebraic forms of the likelihood and the prior are multiplied to produce the posterior, the product can be arranged such that the posterior is of the same distribution family as the prior, and the only integration remaining is across one full probability distribution (equal to 1).

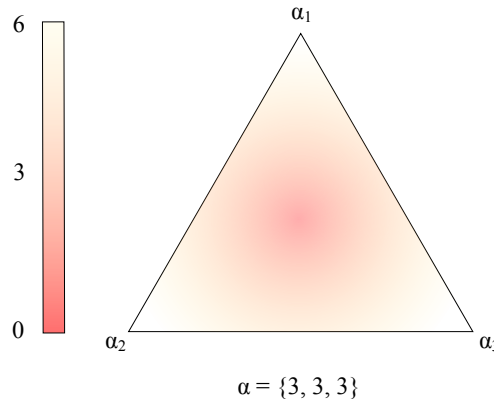


Figure 8: Sample set of hyperparameters for a Dirichlet distribution and associated probabilities, graphically depicted on a Simplex, where $|\alpha| = 3$.

3.1.3 The Dirichlet distribution in the context of Bayesian neural networks

This part is a carefully adjusted presentation of the succinct & coherent proofs by Carlo Tomasi [Tom14], adapted to fit aforementioned syntax, for easier reading.

The PDF of the Dirichlet, which takes a parameter $\boldsymbol{\alpha}$ and generates a set of probabilities depicted as $\boldsymbol{\theta}$, $|\boldsymbol{\alpha}| = |\boldsymbol{\theta}|$, can be represented as follows:

$$\begin{aligned} \text{Dir}_{\boldsymbol{\theta}}[\boldsymbol{\alpha}] &= P(\boldsymbol{\theta}) = c(\boldsymbol{\alpha}) \prod_{k=1}^K \theta_k^{\alpha_k-1} \quad \text{Where } \alpha_k > 0 \text{ for all } k. \\ c(\boldsymbol{\alpha}) &= \frac{\Gamma(\sum_{k=1}^K \alpha_k)}{\prod_{k=1}^K \Gamma(\alpha_k)} \end{aligned}$$

The Dirichlet can be shown to be the conjugate prior of the categorical distribution:

$$\begin{aligned} \text{Cat}_y[\boldsymbol{\theta}] \cdot \text{Dir}_{\boldsymbol{\theta}}[\boldsymbol{\alpha}] &= P(y \mid \boldsymbol{\theta}) \cdot P(\boldsymbol{\theta}) \\ &= \theta_y c(\boldsymbol{\alpha}) \prod_{k=1}^K \theta_k^{\alpha_k-1} = c(\boldsymbol{\alpha}) \prod_{k=1}^K \theta_k^{\tilde{\alpha}_k-1} \\ &= \frac{c(\boldsymbol{\alpha})}{c(\tilde{\boldsymbol{\alpha}})} c(\tilde{\boldsymbol{\alpha}}) \prod_{k=1}^K \theta_k^{\tilde{\alpha}_k-1} = \frac{c(\boldsymbol{\alpha})}{c(\tilde{\boldsymbol{\alpha}})} \text{Dir}_{\boldsymbol{\theta}}[\tilde{\boldsymbol{\alpha}}] \end{aligned}$$

$$\begin{aligned} \tilde{\boldsymbol{\alpha}} &= [\tilde{\alpha}_1, \dots, \tilde{\alpha}_K] \quad \tilde{\alpha}_k = \alpha_k + \delta_{ky}. \\ \delta_{ky} &= \begin{cases} 1 & \text{for } k = y \\ 0 & \text{otherwise} \end{cases} \end{aligned}$$

An unnormalised posterior can be represented as follows, in the context of a training dataset D consisting of tuples (x_i, y_i) :

$$\begin{aligned} P(\boldsymbol{\theta}) \prod_{i=1}^{|D|} P(y_i \mid \boldsymbol{\theta}) &= \text{Dir}_{\boldsymbol{\theta}}[\boldsymbol{\alpha}] \prod_{i=1}^{|D|} \theta_{y_i} \\ &= \text{Dir}_{\boldsymbol{\theta}}[\boldsymbol{\alpha}] \prod_{k=1}^K \theta_k^{\sum_{i=1}^{|D|} \delta_{ky_i}} \\ &= \frac{c(\boldsymbol{\alpha})}{c(\boldsymbol{\alpha}')} \text{Dir}_{\boldsymbol{\theta}}[\boldsymbol{\alpha}'] \end{aligned}$$

$$\boldsymbol{\alpha}' = [\alpha'_1, \dots, \alpha'_K]^T \quad \text{with} \quad \alpha'_k = \alpha_k + \sum_{i=1}^{|D|} \delta_{ky_i} \quad \text{for } k = 1, \dots, K$$

The above process essentially considers the Bayes rule likelihood, represented in terms of the product of the outputs of the likelihood's underlying function (e.g. neural network), and then maps these outputs to the outputs of a Dirichlet PDF by grouping the identical outputs via a summation of Kronecker Delta function outputs, $\sum_{i=1}^{|D|} \delta_{ky_i}$, under the condition that the K parameters of the Dirichlet, where $K = |\boldsymbol{\alpha}|$, are adjusted based on the quantities of the output groups.

As a result of Axiom 2, Proof 1, and the above derivation of the unnormalised posterior, the

evidence can be represented as:

$$\begin{aligned}
P(D) &= \int P(\boldsymbol{\theta}) \prod_{i=1}^{|D|} P(y_i \mid \boldsymbol{\theta}) d\boldsymbol{\theta} \\
&= \int \frac{c(\boldsymbol{\alpha})}{c(\boldsymbol{\alpha}')} \text{Dir}_{\boldsymbol{\theta}}[\boldsymbol{\alpha}'] d\boldsymbol{\theta} \\
&= \frac{c(\boldsymbol{\alpha})}{c(\boldsymbol{\alpha}')} \int \text{Dir}_{\boldsymbol{\theta}}[\boldsymbol{\alpha}'] d\boldsymbol{\theta} \\
&= \frac{c(\boldsymbol{\alpha})}{c(\boldsymbol{\alpha}')}
\end{aligned}$$

The posterior is therefore:

$$\begin{aligned}
P(\boldsymbol{\theta} \mid D) &= \text{Dir}_{\boldsymbol{\theta}}[\boldsymbol{\alpha}'] \frac{c(\boldsymbol{\alpha})}{c(\boldsymbol{\alpha}')} \cdot \frac{c(\boldsymbol{\alpha}')}{c(\boldsymbol{\alpha})} \\
&= \text{Dir}_{\boldsymbol{\theta}}[\boldsymbol{\alpha}']
\end{aligned}$$

As a result, the posterior depends solely on the training dataset and the prior. The predictive distribution is then:

$$\begin{aligned}
P(y \mid D) &= \int P(y \mid \boldsymbol{\theta}) \cdot P(\boldsymbol{\theta} \mid D) d\boldsymbol{\theta} \\
&= \int \text{Cat}_y[\boldsymbol{\theta}] \cdot \text{Dir}_{\boldsymbol{\theta}}[\boldsymbol{\alpha}'] d\boldsymbol{\theta} \\
&= \int \theta_y \text{Dir}_{\boldsymbol{\theta}}[\boldsymbol{\alpha}'] d\boldsymbol{\theta} \\
&= \int c(\boldsymbol{\alpha}') \prod_{k=1}^K \theta_k^{\alpha'_k + \delta_{ky}} d\boldsymbol{\theta} \\
&= \frac{c(\boldsymbol{\alpha}')}{c(\boldsymbol{\alpha}'')} \int \text{Dir}_{\boldsymbol{\theta}}[\boldsymbol{\alpha}''] d\boldsymbol{\theta} & \alpha''_k = \alpha'_k + \delta_{ky} \\
&= \frac{c(\boldsymbol{\alpha}')}{c(\boldsymbol{\alpha}'')}
\end{aligned}$$

The above repeats the use of $\boldsymbol{\alpha}'$ as seen when deriving the unnormalised posterior, but adds an extra count for the data point y under test, via a Kronecker Delta function, δ .

This can then be simplified to [Min00]:

$$P(y \mid D) = \frac{N_y + \alpha_y}{N + \sum_y \alpha_y} \quad (6)$$

This essentially means the summation of the quantity of data points of the category of y , plus the parameter α_i associated with category y , divided by the summation of all α_i , $\alpha_i \in \boldsymbol{\alpha}$, and the N quantity of data points of all categories.

As a result, the values of EU and AU are directly proportional to the dataset and the

parameters in the prior, which may be represented as the tuple (D, α) . Furthermore, the IHDR can be calculated for each instance of (D, α) via a standard sort algorithm among the probabilities of the K categories, $K = |\alpha|$.

3.1.4 The Julia programming language

Although neural networks can make very accurate predictions, at their core they are identifying complicated patterns in data, patterns of which may be difficult to comprehend by humans, as a result of the scale of neurons within a modern neural network. As a result, it is expected that more manual methods of processing data may continue to be of value, for example to assess if the underlying data distribution that a neural network was trained upon has changed, by comparing predictions over time.

Julia is a programming language designed for high performance statistical analysis, which is simultaneously receiving first party support from major SIMD manufacturers, alongside machine learning Python libraries. Furthermore, the Julia programming language has shown comparable performance with the less domain specific and low level programming language C and the OpenMP multi-core library.[GVLD⁺23] For this reason the Julia programming language will be explored.

3.1.5 Rudimentary experiment

Consider a static dataset of K categories, each of quantity $|d|$, $d \in D$, the posterior and therefore the EU and AU, will then be entirely dependent upon α , where $|\alpha| = K$. There is no way to variate the model, as in $exP_{likelihood}$, in the context of the conjugate Dirichlet prior. As a result, it would be expected that the maximum a posteriori will be dependent upon the ratios of the values within α . Specifically, this will involve iterating over a set of permutations of α , generating an associated probability of y occurring, and then graphing them.

Minimalist permutation generator Consider that we want to iterate over 2^{m^n} values of α , where $m \cdot n \leq s$, and the maximum integer size is $2^s - 1$, to stay within the realm of integers. The simplest way is to choose a desired value n , whereby $n < s$. This will mean that m bits can be allocated for each $\alpha \in \alpha$, where $m = s \text{ div } n$, and therefore that the permutation function can repurpose hardware arithmetic and the assembly AND operation.

Take $m, n, |d|, |D| \in \mathbb{N}$. The general algorithm is then as follows.

Algorithm 1 *Permutation generator*

1. Define $m, n, |d|$, where $d \in D$
2. $i = 0$
 - (a) $j = 0$

- (b) $\alpha = 0$
- (c) $target\text{-}bits = 2^m - 1$
 - i. Shift $target\text{-}bits$ leftwards $j \cdot m$ times
 - ii. $\alpha = target\text{-}bits_2$ AND i_2 , shift rightwards $j \cdot m$ times
 - iii. $\alpha_j = \alpha$
 - iv. $j = j + 1$
 - v. If $j = n$ break loop, else loop
- (d) Calculate $P(y | D)$, store in S_y
- (e) $i = i + 1$
- (f) If $i = 2^s - 1$ break loop, else loop

3. Return S_y

The Appendix features an example of permutation output where $(m, n) = (4, 3)$.

The constraint of this algorithm is that generating $2^{m \cdot n}$ permutations, each of which feature $m \cdot n$ bits, can be computationally expensive, and potentially also memory intensive, if stored. Furthermore, the condition $d > 0$ must be met, due to the division within $P(y | D)$.

It is possible to calculate $P(y | D)$ (equation 5) either via division or in a base-2 logarithmic form. Fortunately Julia offers very easy timing capabilities, and in the instance that the tuple (m, n, d) are set to $(4, 6, 1)$, the following was observed, during the 2nd run (Julia documentation discourages benchmarking 1st runs):

- 2.054777s when performing subtraction between log base-2 floats
- 1.811998s when performing float division of 2 integers

In the case that x GB is available, this permits $x \cdot (1000)^3 \cdot 8$ bits of available space. If only the probabilities of $P(y | D)$ are stored, consider that $2^{m \cdot n}$ is equivalent to $2^{m \cdot n}$, such that the following holds:

$$\log_2(x \cdot (10)^9 \cdot 8) \geq m \cdot n$$

Unfortunately Julia does crash when trying to plot the case in which (m, n, d) is equal to $(4, 6, 1)$, and browser issues reject the quantity of permutations generated by $(4, 5, 1)$.

3.1.6 Results

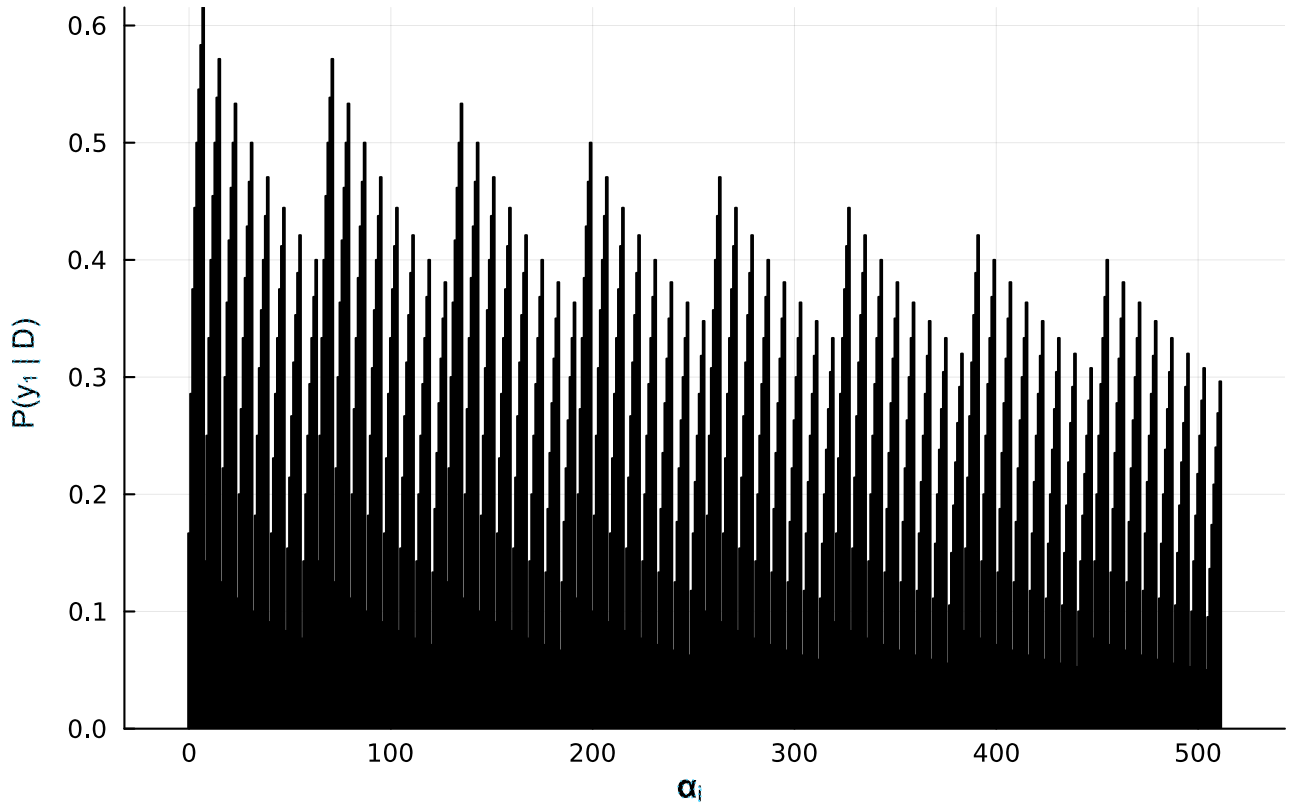


Figure 9: sample bar graph to show how probability of category y_1 changes, $P(y_1 | D)$, in the case of on one specific instance of training data D , as α_i changes, in the case that (m, n) is equal to $(3, 3)$, recalling that $|\alpha| = n$. Note that α_i is multidimensional; this is a 1-dimensional representation force fit.

In the case of the Dirichlet distribution, entropy is calculated as follows:

$$h(\mathbf{X}) = \ln B(\alpha) + (\alpha_0 - K)\psi(\alpha_0) - \sum_{j=1}^K (\alpha_j - 1)\psi(\alpha_j)$$

Entropy extreme	Entropy value	α
Highest	-0.6931	[0x1, 0x1, 0x1]
Lowest	-2.6239	[0x8, 0x1, 0x1]

Table 1: Dirichlet entropy values, for all permutations of α where $(m, n) = (3, 3)$

Uncertainty type	Value
Aleatoric	-2.6239
Epistemic	1.9307
Total	-0.6931

Table 2: Uncertainty values, for all permutations of α where $(m, n) = (3, 3)$

3.2 Experiment #2: Architecture Size

3.2.1 Purpose

This experiment sets up the basic experiment environment that future experiments are then built off of, by making iterative adjustments to it. This experiment therefore features a simple, deterministic model and training process, with only customisations that increase the simplicity and ease of interpretability of the results (e.g. a training dataset that can be represented in one image).

3.2.2 Methodology

Ideally a general purpose library that implements variational inference in the context of Bayesian neural networks and successfully fulfills the theory behind the original Bayes-by-backpropagation paper would be available, such that CBDL can be demonstrated at scale. Initially, however, the implementations themselves are to be tested.

Model To focus solely on the process, and less on the output, the prediction capabilities of the model will be a simple probabilistic binary classifier. A simple and numerical example would be classifying whether a number is an approximate representation of π or not, a common task for calculators during algebraic representations, and this case easily fits a Gaussian distribution, whereby μ is the best approximation of π that fits the floating-point data structure available, $r \approx 3\sigma$ represents the range, and $[\pi - r, \pi + r]$ the range that is accepted as π . A neural network of architecture $\{1, n, n, 1\}$, where $n \in \mathbb{N}$ will be trained.

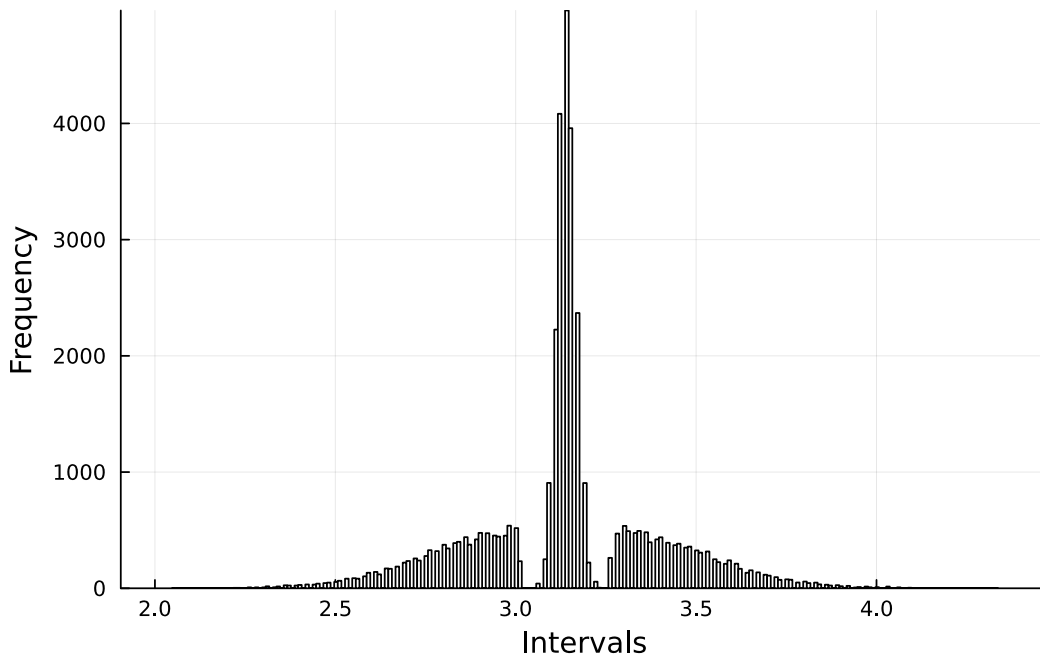


Figure 10: Bar chart, reminiscent of a Laplace distribution (or a Dubai cityscape), to show dataset generated via three Gaussian distributions, the output of two being modified via a modulus function, but all calibrated around the value of π .

Dataset First, a dataset needs to be generated that meets the requirements. Figure - depicts a dataset generated via Gaussian distributions, one of which is associated with the valid π class, the other two with the invalid π class.

Loss function Next, an appropriate binary classification loss function is to be selected, which is the Binary Cross Entropy loss function, a PyTorch built-in.

Finally, the architecture of the model is to be specified. To make it as easy to inspect the model parameters process, for use in a later experiment, it is desirable to keep the model as small as possible, whilst simultaneously large enough to be trainable in a short period of time.

3.2.3 Results

Figure - shows the average loss from the test set, for different models made up of the same linear and ReLU layering sequence, of architecture $\{1, n, n, 1\}$, where n represents the quantity of neurons in the hidden layer, $n \in \{32, 128, 256, 512, 1024, 2048\}$, using a training dataset of 40,000 numbers, 20,000 of which are to be classified as π (class 1). The values of θ , in respect to all linear layers of all models, were initialised with a uniform distribution $U(-0.25, 0.25)$. The datasets were shuffled using a constant random seed. Training was run for 300s minimum (until the epoch running at the 300s mark completed), to gauge which architecture produced the lowest average loss fastest. The models were trained via CPU, where the quantity of cores was greater than the training instances running, whilst the machine was otherwise idle.

The data indicates that there is a strong correlation between scaling upwards the size of the hidden layer and prediction accuracy, when controlling for all other factors. Note that the model where $n = 1024$ took 400s for 2 epochs, and the model where $n = 2048$ took 1250s for 1 epoch.

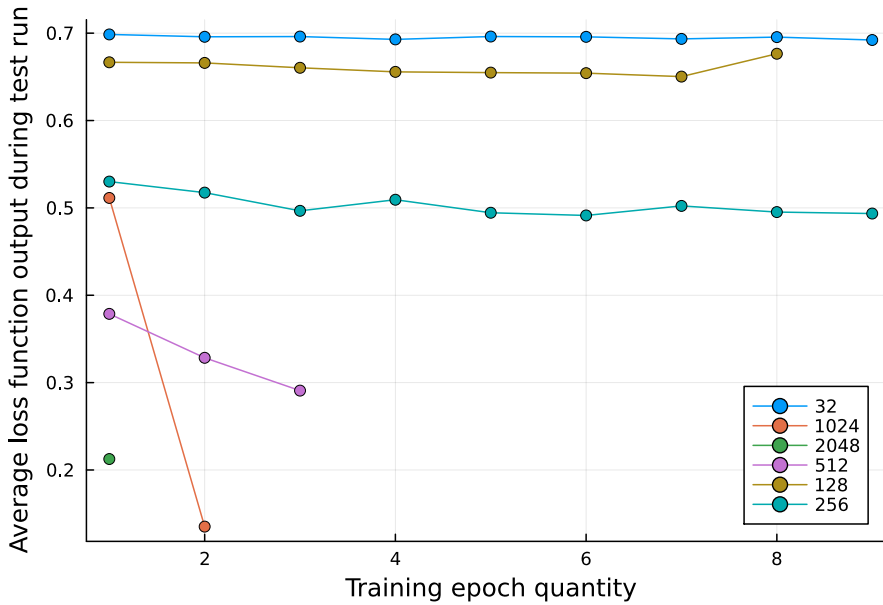


Figure 11: Line graph to show how changes in scale of model architecture lead to changes in average loss function output when comparing output with test dataset, after training different architectures for the same quantity of time.

3.3 Experiment #3: Reparameterisation

3.3.1 Purpose

This experiment will instead involve investigating how the theory of Bayes-by-backpropagation can fit with a major ML library, specifically PyTorch, due to being strongly intertwined with open-source models. The *backward()* function available within PyTorch is able to find $\frac{\partial L}{\partial \xi}$ for any model parameters ξ , such as if values of θ_i , $\theta_i \in \theta$, are generated via a function featuring ξ . This is done by automatically tracking state changes via graph theory, and numerical derivatives, as in Figure .

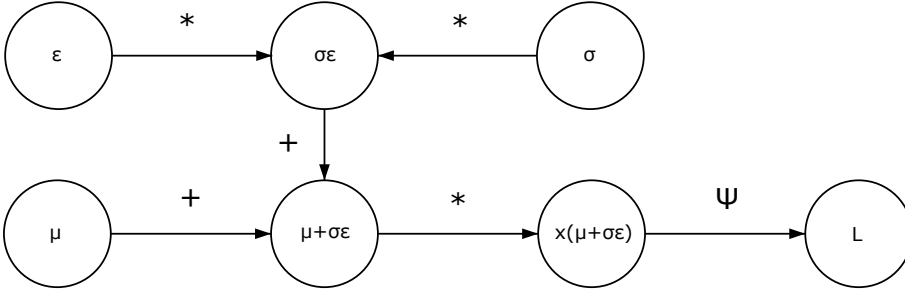


Figure 12: Diagram to show how partial derivatives can be calculated from the loss function L to the leaf nodes, containing σ , μ , which are elements of ξ , via graph theory, in the context of basing θ , on Gaussian distributions.

As a result of the above, samples of new values of θ can be generated during each forward propagation, akin to sampling from the relevant probability distribution.

3.3.2 Methodology

Model Due to the results of Experiment #2 exhibiting the efficacy of simply increasing model size for the previously described binary classification task, the architecture selected for this experiment is of sequence $\{1, 1024, 1024, 1\}$.

Dataset 50,000 instances of n , $n \in \{\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2} \mid n \in \mathbb{R}\}$, generated via three Gaussian distributions, one for which $\mu = \pi$ and for which samples from are to be classified as π .

Hyperparameters To stay similar to initialising with a distribution of $U(0.25, 0.25)$, instead a distribution of $N(0.25, 0)$ is utilised.

VI implementation The original Bayes-by-backpropagation paper encourages σ to be generated by another parameter ρ , via $\sigma = \ln(1 + \exp(\rho))$ where $\sigma \in \xi$ in the case that Gaussian distributions are assigned to elements of θ , to prevent σ becoming negative. However, choosing the wrong value of ρ can quickly lead to exploding gradient (inadvertently exhibiting the value of CDBL), which then causes all data points to be classified the same. The initial values of (μ, ρ) were selected via trial-and-error, to suit this context, when accounting for $\rho = \ln(e^\sigma - 1)$. PyTorch can then automatically handle iterations to (μ, ρ) associated with each θ_i , $\theta_i \in \theta$.

Loss function During training, the KL-divergence loss function, which relates the probability of the model parameters to the model, can be based on the Laplace distribution, due to the similarity to the underlying data distribution. That is to say, $P(D | \theta)$ is calculated by hypothesising that the dataset's numbers to be considered acceptable as π follows a Laplace distribution, as suggested in Figure .

$$L(\hat{y}, y) = \begin{cases} \text{Laplace}_{CDF}(\hat{y}, \pi, b) & \text{for } y = 1 \\ 1 - \text{Laplace}_{CDF}(\hat{y}, \pi, b) & \text{for } y = 0 \end{cases}$$

This can then be modelled as a more typical binary cross-entropy loss function, with a minor modification:

$$L_{BCE} = \sum_{i=1}^n [y_i \cdot \ln\{P(\hat{y}_i \in A)\} + (1 - y_i) \cdot \ln\{P(\hat{y}_i \notin A)\}] \quad y_i \in \mathbb{B}$$

$$L_{BCE\text{Laplace}} = \sum_{i=1}^n [y_i \cdot \ln\{\text{Laplace}_{CDF}(\hat{y}_i)\} + (1 - y_i) \cdot \ln\{1 - \text{Laplace}_{CDF}(\hat{y}_i)\}]$$

However, as the Laplace CDF is a piecewise function, it is problematic to fit into the back-propagation procedure when processing data in batches. Therefore, the parameterised Sigmoid is used as a model of the Laplace CDF; the venerable Sigmoid output layer of Experiment #2 remains. There does not appear to be a straightforward algebraic relation between the Laplace b and the Sigmoid k that can be implemented during model Ψ initialisation, though there may be a numerical method that is faster than training the neural network from a generic initial value of k .

$$\text{Sigmoid}(x, k) = \frac{1}{1 + e^{-kx}}$$

$$\text{Laplace}_{CDF}(x, 0, b) \approx \text{Sigmoid}(x, k)$$

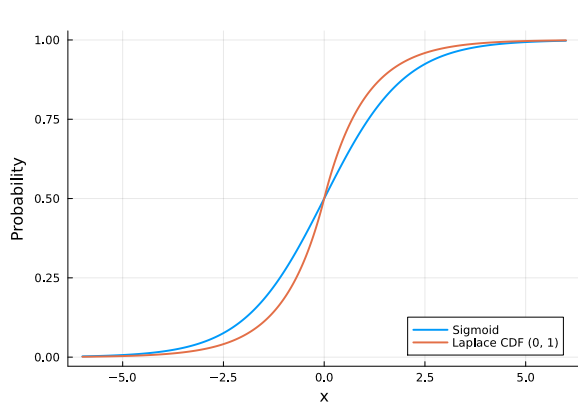


Figure 13: Line graphs of Sigmoid, and Laplace CDF where $(\mu, b) = (0, 1)$.

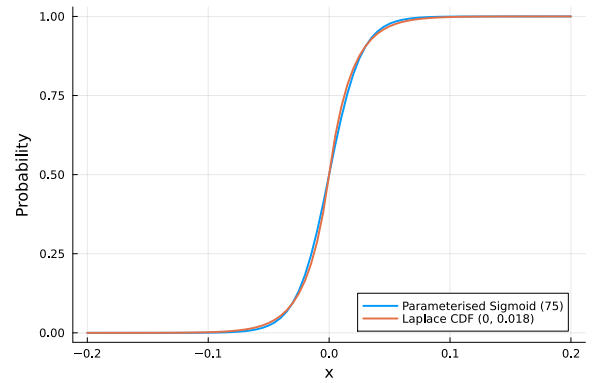


Figure 14: Line graphs of parameterised Sigmoid where $k = 75$, and Laplace CDF where $(\mu, b) = (0, 0.018)$.

In initial trials, k was found by inspection, by comparing the underlying distribution (Figure .) and calibrating the Laplace PDF (μ, b) to it, with k set as a trainable parameter. However, this

resulted in issues of underflow/overflow. Under-the-hood, PyTorch uses a logarithmic Sigmoid function, difficult to customise - see `Activation.cpp` for some cumbersome behind-the-scenes C++ including “Workaround for gcc-14.2.0”. Eventually, defeat was accepted regarding custom output layers, and a typical Sigmoid function was deployed, under the expectation that the linear layers, with reparameterisation implemented, are effective enough alone to split the data.

Training environment To find the best training environment for upcoming experiments, a variety of batch sizes and hardware was tested, to assess the effect on training time.

3.3.3 Hypothesis

Recall that we cannot possibly sample all possible values of θ . However, once the model is trained, it is possible to take a quantity of n samples of θ , an approximation (contextually may be valid or invalid) of π , the trained model, Ψ , and run the approximation through the model n times, then convert the classification results to a probability, which can be averaged. This can be repeated for a wide interval. These probabilities should graph to a model distribution which approximates that of the generation process. For example, a probability of 0.5 will ideally be generated via a data point exactly on the boundary of the 2 classes, and similarly appropriate probabilities for the whole predictive distribution.

3.3.4 Results

Table 3: How duration of one training epoch changes with batch size. Note that all the figures regard a training dataset of size 40,000, and a test dataset of size 8,000.

Hardware	Batch size	Duration (s)
2x L40S	1	100.1658
2x L40S	2	86.01277
2x L40S	4	43.9597
2x L40S	8	22.4844
2x L40S	16	11.7968
2x L40S	32	6.4417
2x L40S	64	3.7610
2x L40S	128	2.5118
2x L40S	256	1.8248
2x L40S	512	1.4977
2x L40S	1024	1.0099
2x A100	128	3.3116
2x A100	256	2.4063
2x A100	512	1.9498
2x A100	1024	1.4774
1x EPYC core	256	101.2247

Optimisation of model parameters ξ is performed per batch. Therefore, in the case that sampling is performed per-batch, to ensure that the probability distribution parameters ξ eventually generated are as fitting as possible, it may be desirable to maximise the sampling, by lowering the batch size as much as possible, or increasing the training dataset as much as possible, so long as the training duration is tolerable.

Unfortunately, however, the model parameters ρ of ξ , which are used to generate σ , were not iterated by PyTorch, seemingly the automatic differentiation procedure was thrown off by the $\sigma \cdot \epsilon$ product, $\epsilon \sim N(0, 1)$, as discussed in the Bayes-by-backpropagation section of the theory. This potentially means that PyTorch is incompatible with Bayes-by-backpropagation, despite attempting implementation of the sampling and arithmetic in line with the original paper.[BCKW15]

The following graphs nonetheless demonstrate the probability distributions that could be generated via the deterministic models that were trained. The noisy gradient is a result of the epsilon sampling, from $N(0, 1)$, being added to the weights.

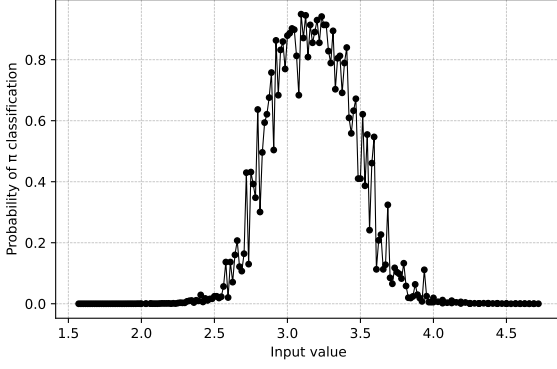


Figure 15: Plot of probabilities of n , $n \in \{\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2} \mid n \in \mathbb{R}\}$ being classified as π , after 1 training epoch.

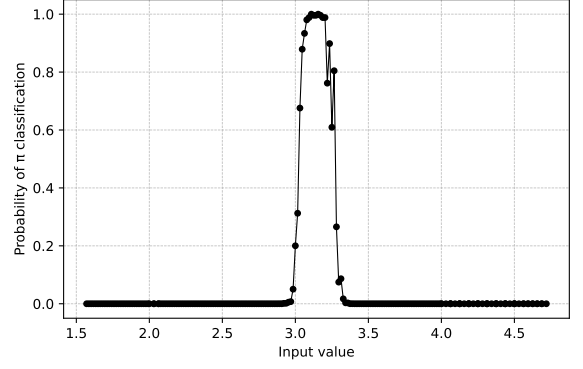


Figure 16: Plot of probabilities of n , $n \in \{\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2} \mid n \in \mathbb{R}\}$ being classified as π , after 128 training epochs.

3.4 Experiment #4: Draft CBDL

3.4.1 Purpose

At this point the following hypotheses have not yet been rejected:

- Scaling a neural network can instantly improve predictive capabilities
- Modern neural network libraries have been designed for deterministic neural networks
- Probability distributions can be built via modern neural network libraries, within a limited context

The issue of ambiguity that CBDL aims to address is that it is easy to infinitely scale a neural network upwards in parameters, ξ , but more difficult to maintain the same balance in the proportion of training data or training epochs available as scaling occurs. These constraints of larger scale models can be simulated at smaller scale, so as to induce epistemic uncertainty, and the general approach should then be applicable to a larger scale.

Whereas the previous experiment aimed to fit the theory to the available implementations, this experiment will aim to do the opposite, by exploring what is currently available in the field of probabilistic modeling.

3.4.2 Methodology

Model A neural network of architecture $\{1, 1024, 1024, 1\}$ will be trained.

Dataset 50,000 instances of n , $n \in \{\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2} \mid n \in \mathbb{R}\}$, generated via three Gaussian distributions, one for which $\mu = \pi$ and for which samples from are to be classified as π .

Epistemic uncertainty To induce epistemic uncertainty (i.e. cause the model to underfit the training data), the following approaches can be taken:

- Increase quantity of parameters of model
- Reduce training epochs run
- Reduce available training data

Once epistemic uncertainty is induced, it then becomes a matter of how to measure it. CBDL takes the approach of a variety of initialisation methods for θ_i , via exP_{prior} , and a variety of models, Ψ , via $exP_{likelihood}$. This approach can be somewhat approximated using an ensemble of deterministic models, provided the trained models generate different PDFs.

Draft CBDL It could be argued that deterministic networks skip the quintessential $P(\theta | D)$ step. Alternatively it could be considered that the trained instances of θ_i are akin to samplings from $P(\theta | D)$, albeit where $P(\hat{y} | \theta) \cdot P(\theta | D)$ is then of an unknown probability.

Furthermore, it could be modeled that if n instances of θ_i are initialised via a uniform prior distribution $U(a, a)$ and trained, then samples from any given prior distribution $P(\theta)$ are highly unlikely to ever perfectly match the multidimensional, continuous space that θ_i exists within - a uniform prior - and therefore it could be modeled that all trained instances of θ_i are equally probable, i.e. $P(\theta_i | D) = \frac{1}{n}$. This could be described as a uniform reduced posterior, reduced in that it is made up only of the more probable values of $P(\theta | D)$, and therefore the possible values of θ_i that can be generated from it is reduced.

If Ψ_{θ_i} then represents a probability distribution for a given interval that $\hat{y}$ resides in, then $P(\hat{y} | \theta_i)$ can simply be generated by the model. Therefore:

$$P(\hat{y} | \theta_i) \cdot P(\theta_i | D) = \Psi_{\theta_i}(\mathbf{x}) \cdot \frac{1}{n}$$

Alternatively, it may be possible to generate a uniform prior by sampling from a Gaussian distribution, $N(0, \sigma^2)$, with a large value of σ .

Training environment In Experiment #4, due to the scale, the instances of the model are more suitable to GPUs, via clustered computing. Batch jobs are generated via command-line arguments, and the weights generated collected at a later point in time. Simultaneously, to reduce energy consumption, a limited selection of model sizes and prior.

Dataset 50,000 instances of n , $n \in \{\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2} \mid n \in \mathbb{R}\}$, generated via three Gaussian distributions, one for which $\mu = \pi$ and for which samples from are to be classified as π .

3.4.3 Hypothesis

It may be possible to simulate a uniform prior via a series of uniform distributions, of shape $U(a, a)$. Alternatively, via a Gaussian distribution of $N(0, 1)$, such as the Kaiming variation.[HZRS15] A uniform prior can then resemble samples from an unknown posterior, $P(\theta | D)$, via deterministic training.

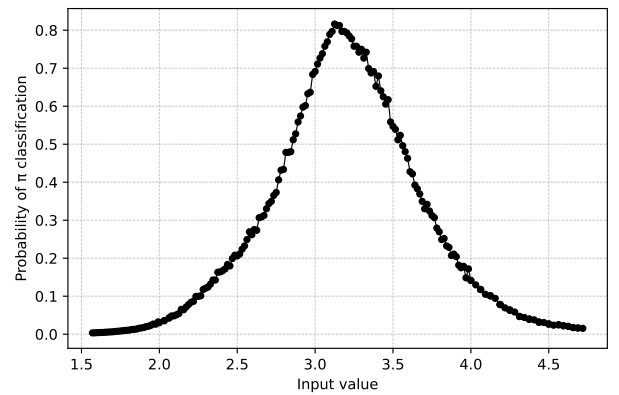
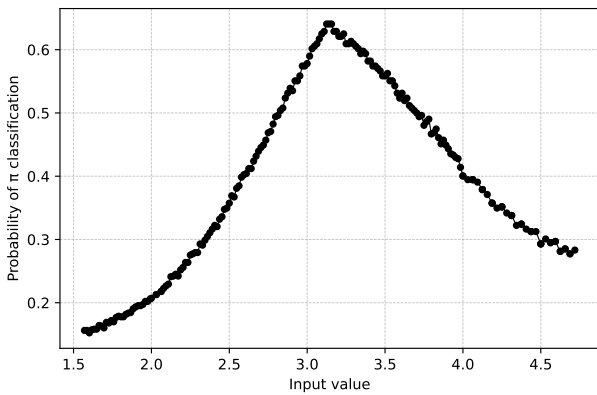
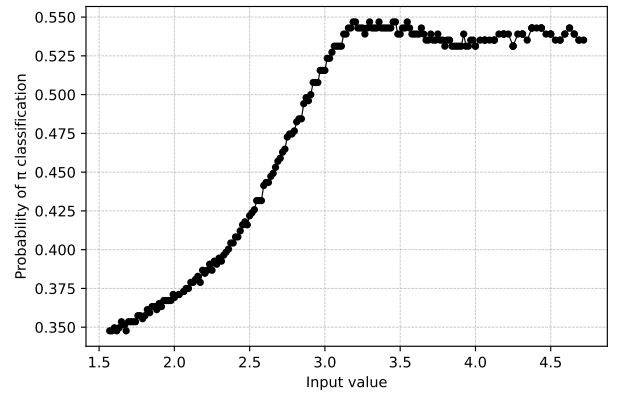
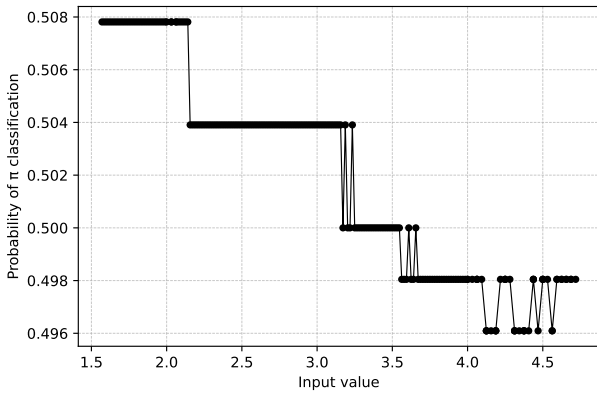
3.4.4 Results

When the interval between values of a , $a \in \mathbb{R}^+$, of the uniform distributions used, $U(a, a)$, is large enough, then the prior does not appear to be uniform. Specifically, the loss function output after 5 training epochs, when controlling for other hyperparameters, differs significantly, as shown in the following table. A batch size of 1024 was specified.

Table 4: Binary cross-entropy loss output changes per epoch, when model parameters are initialised via different uniform distributions.

$U(a, a)$	Epoch #1	Epoch #2	Epoch #3	Epoch #4
$a = 0.25$	0.621	0.589	0.564	0.548
$a = 0.5$	4.492	20.406	0.469	0.141

Generalisation of the above results can be visualised at scale in the following graphs, where $a \in \{0.0625, 0.1875, 0.25, 0.375, 0.4375, 0.5\}$, and the quantity of training epochs is fixed, controlling for dataset and model, including deterministic random seed preshuffling.



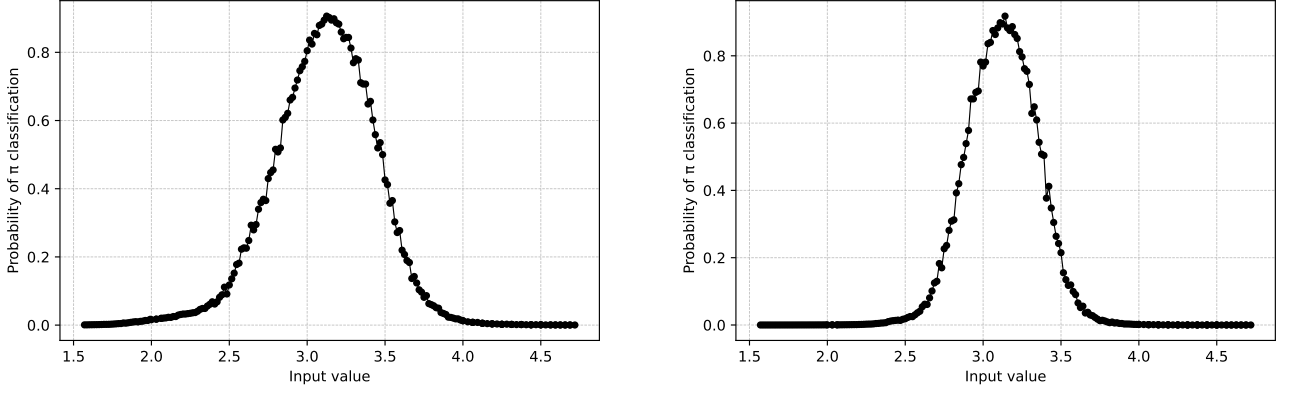


Figure 17: Six line graphs of probabilities of n , $n \in \{\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2} \mid n \in \mathbb{R}\}$ being classified as π , after 50 training epochs, when model is initialised with weights from distribution $U(a, a)$, where $a \in \{0.0625, 0.1875, 0.25, 0.375, 0.4375, 0.5\}$, graphs ordered by ascending a .

Initialising parameters via a Kaiming Gaussian distribution method resulted in a wide variety of initial loss function outputs, again leading to the existence of an easily accessible uniform prior, based on non-iterative probabilistic sampling, being inconclusive.

Table 5: Binary cross-entropy loss output after 1 epoch, when model parameters θ are sampled from a Kaiming Gaussian distribution, but with different random seeds set beforehand.

Random seed, for Kaiming Gaussian sampling	Initial loss function output
33958	1.039
54600	2.357
117643	0.869
493644	0.739
525075	1.374
600768	1.811
754165	1.602
757191	1.794
795914	1.788
958536	16.344
86079	3.707

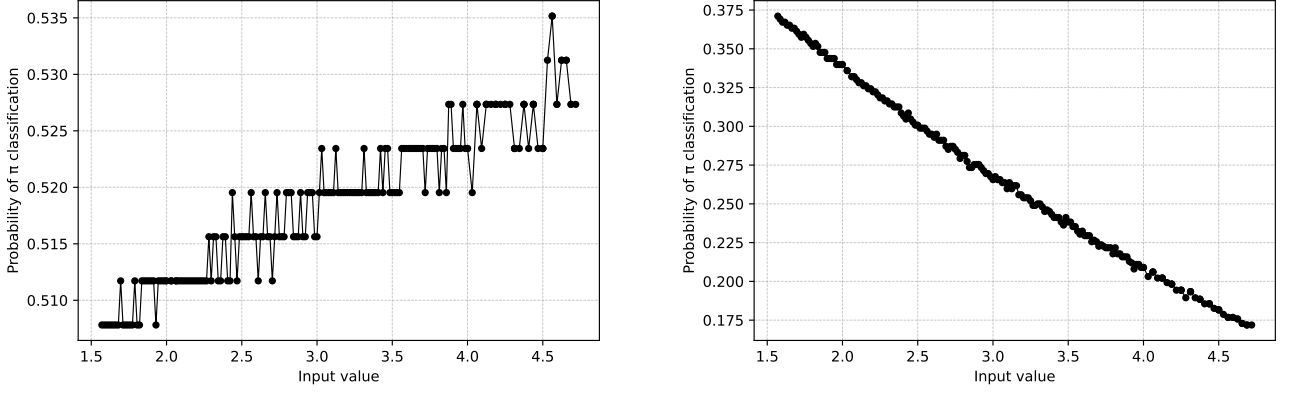


Figure 18: Two line graphs of probability distributions generated, after 150 training epochs, with batch size of 1024, when controlling for model architecture, when parameters θ are initialised via Kaiming Gaussian distributions sampled with different random seeds, (114265, 659475).

The above data demonstrates the sensitivity of resulting values of θ , in the context of either data scarcity or compute scarcity, and therefore the reliance on the prior distribution, in the case of high epistemic uncertainty.

These investigations into uniform distributions and Kaiming Gaussian distributions also exhibit the difficulty of selecting a uniform prior in the context of neural networks.

3.5 Experiment #5: Reparameterisation Retry

3.5.1 Purpose

After a brief foray into a broken MCMC implementation that initially suggested promise, BN-NPrior [FGAvdWA21], it was desirable to attempt Bayes-by-backprop once more, despite the previous implementation issues, via a different approach.

It seemed that the only alternative to Bayes-by-backpropagation would be to train a large quantity of deterministic models (a Bayesian ensemble), adjusting the prior and likelihood via basic methods such as:

- Dropout, or architecture variation
- Parameter initialisation selection
- Adding noise during gradient descent (SGLD) [MCO17]

Ultimately each trained model would then represent one point within a multidimensional posterior, making ensembles a computationally expensive option.

3.5.2 Methodology

Model A neural network of architecture $\{1, 1024, 1024, 1\}$ will be trained.

Dataset 50,000 instances of n , $n \in \{\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2} \mid n \in \mathbb{R}\}$, generated via three Gaussian distributions, one for which $\mu = \pi$ and for which samples from are to be classified as π .

LLM implementation generation Major LLM Gemini 3.5 was used to generate 3 functions related to Bayes-by-backpropagation, and this implementation proved to be dramatically simpler than the 3rd-party library Bayesian-Torch that Experiment #3 had been inspired by, after careful inspection, commenting, and extraction. Simultaneously, thorough understanding of the Bayesian-Torch library made the new implementation easy to comprehend and tweak. This new implementation leaned much more heavily on PyTorch built-ins, such as `.rsample()`, the official PyTorch reparameterisation sampling method, and calculated and stored the KL-loss in a way that would be easy to access, unlike the approach seen in Bayesian-Torch which outputs the KL-loss from each layer during the forward pass. The prompts used to generate this implementation can be found in the official repo of this paper. However, it was then found that the KL divergence implementation was incompatible with GPU processing, and so modifications had to be made, including outputting it alongside $\hat{y}$, until eventually a functioning recipe of PyTorch instructions was deduced.

Precision In previous experiments, only the data type *bfloat16* was set, meaning only 16 bits, $\{1, 8, 7\}$ (sign, exponent, mantissa) were available for each element within ξ . In this experiment, *float64* and *float32* were also investigated, to reduce the chance of the vanishing gradient problem.

Loss function It was now possible to incorporate the KL divergence into the loss function. A typical deterministic loss function involves the mean or summation of loss function outputs across a given batch size.

The ELBO’s $\mathbb{E}_{q_\xi}[\log P(D|\theta)]$ could be implemented either as:

- Summation (proportional to batch size)
- Mean (in terms of batch size)

The ELBO’s $\text{KL}(q_\xi(\theta) \parallel P(\theta))$ is not affected by batch size, but could either be implemented either as:

- Summation of KL divergence metrics, one metric per θ_i , across whole model
- Mean of KL divergence metrics, across whole model

The KL divergence function is unbounded positively, $[0, \infty)$, for any one instance, unlike the instances of log probabilities in $\mathbb{E}_{q_\xi}[\log P(D|\theta)]$.

Furthermore, unless architectures and batch sizes have been optimised within the constraints of hardware, for example in the context of highly scaled implementations, there is not generally any expected proportion between batch size and parameter quantity $|\theta|$, nullifying the value of summation of each. Therefore, the mean of the likelihood probabilities and the mean of the KL divergence metrics make for a more appropriate combination. This also allows for comparing across batch sizes and architectures more consistently.

The KL divergence is recalculated per batch, by summing the metrics across all layers, and then converting to a mean via quantity of model parameters:

$$L_{KL} = \frac{\sum_{i=1}^N \sum_{j=1}^{|\theta_l|} \text{KL}(q_{\xi}(\theta) \parallel p(\theta \mid D))_j}{|\theta|}$$

Where N is the quantity of layers, b is the data batch size, $|\theta_l|$ is equal to the quantity of parameters within 1 layer. $|\theta|$ can be calculated via $\sum_{N-1} m \cdot n$, where m and n represent the quantity of input features in each adjacent pair of layers $(m, n)_i$ of the neural network, Ψ .

The above formulation means every KL divergence metric is of equal value.

Environment The original development environment was CPU-based, but, unfortunately, once transferred to the batch-based compute cluster to run experiments at scale, various unexpected errors were hit. Therefore, the experiment environment was changed to a commercial Jupyter notebook service that offered GPU runtimes, which was more portable (as in, no errors) between CPU and GPU. This could be due to the PyTorch versions being equivalent, PyTorch 2.10, whereas the compute cluster was running PyTorch 2.6.

Model A deterministic model fallback was also setup, so that the methodology of the trials could incrementally drift onwards from the setup of Experiment #4, such as by introducing repeated sampling (using the same input) during utilisation stage.

3.5.3 Hypothesis

Once ξ has been trained, it should be possible to create a probability distribution by making a large variety of samples for each possible value of n within the interval $\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2}$.

3.5.4 Results

To ensure the transition to the new experiment environment was not detrimental, Experiment #4 was retrofitted and repeated, i.e. a deterministic model. The utilisation code was also adjusted to take multiple samples and then a mean of them, to ensure this worked, before changing the model. Results were as expected.

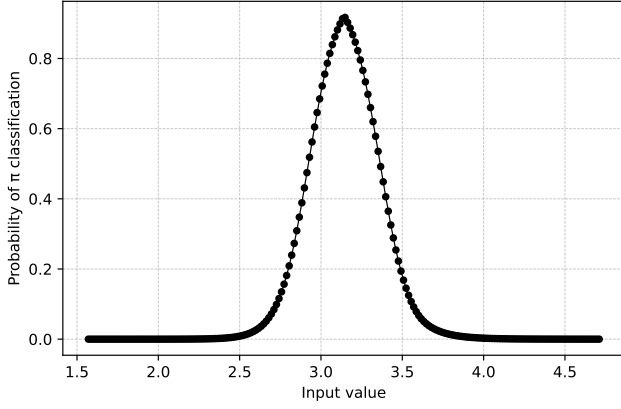


Figure 19: Line graph to show probability distributions generated, of n , $n \in \{\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2} \mid n \in \mathbb{R}\}$ being classified as π , when training deterministic layers, weight initialisation $U(0.1, 0.1)$, training epochs 50, batch size 64.

The reparameterisation implementation was then run. Both KL-divergence updates and iterations of the parameter $\boldsymbol{\rho}$ (that $\boldsymbol{\sigma}$ is generated from) occurred, when precision was changed from *bfloat16* to a 32-bit or 64-bit variant (note that *float16*, as in $\{1, 5, 10\}$ in terms of {sign-bit, exponent, mantissa}, was non-functional).

The next step was to setup values as close as possible to the previous $U(0.1, 0.1)$ trial, and the obvious option seemed to be $N(0.1, 0.000045)$ which was setup via $(\boldsymbol{\mu}, \boldsymbol{\rho}) = (0.1, -10)$, with an equivalent batch size and epochs of 64, 50, respectively. The KL-divergence was also disabled from the loss function, to keep as close as possible to the deterministic equivalent. Unfortunately the loss function output began at 0.693216 but quickly stagnated at 0.693147. The utilisation of the model then resulted in Figure ..

A smaller batch size, 16, was then attempted with $(\boldsymbol{\mu}, \boldsymbol{\rho}) = (0.1, -10)$. Unfortunately this again led to stagnation of the loss function output, at 0.693147. Furthermore, a batch size of 1 quickly stagnates around ~ 0.693 .

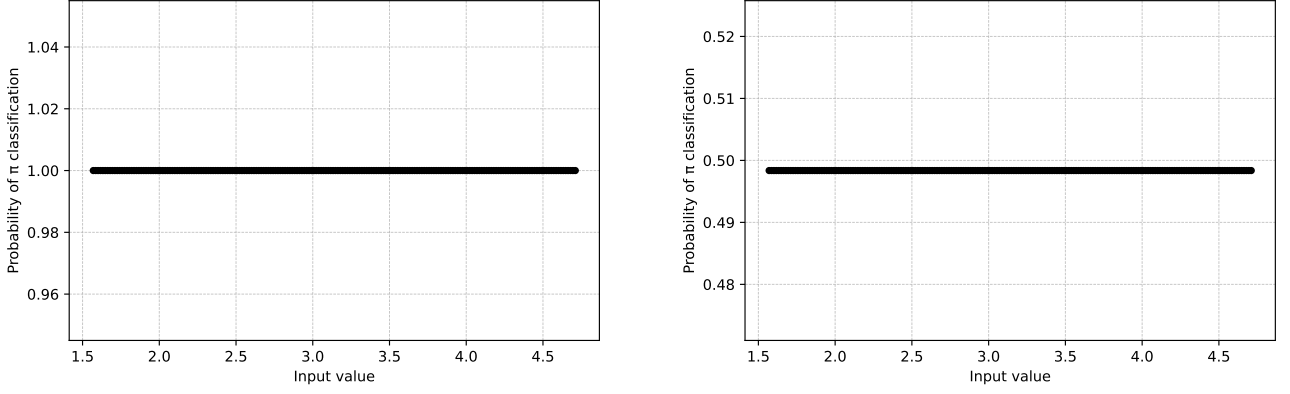


Figure 20: Line graphs to show probability distributions generated, of n , $n \in \{\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2} \mid n \in \mathbb{R}\}$ being classified as π , when training reparameterised layers, where prior $(\boldsymbol{\mu}, \boldsymbol{\rho}) = (0.1, -10)$, batch size is 1, and KL-divergence is not a part of the loss functions. Training epoch sizes are as follows, from left to right: $\{10, 20\}$. Batch sizes are as follows, from left to right: $\{64, 1\}$

Furthermore, when attempting to sum $\boldsymbol{\sigma}$, which is generated via $\boldsymbol{\rho}$, to easily deduce if any values of $\boldsymbol{\rho}$ are iterated during the `backward()` backpropagation pass, different hardware produced different results.

$$\sum_{i=1}^{|\boldsymbol{\sigma}|} \sigma_i = \sum_{i=1}^{|\boldsymbol{\rho}|} \ln(1 + \exp(\rho_i))$$

Dataset batch n	Layer #1	Layer #2	Layer #3	$\sum_{i=1}^{ \boldsymbol{\sigma} } \sigma_i$
1	762.2621	780556.25	762.2621	782080.7742
2	762.2605590820	780556.25	762.2622070312	782080.7728
3	762.2602539062	780556.25	762.2625732422	782080.7728
4	762.2587890625	780556.25	762.2626953125	782080.7715

Table 6: How values of the summation of all elements within $\boldsymbol{\sigma}$ change, during 1st training epoch, when all model parameters are initialised with $(\boldsymbol{\mu}, \boldsymbol{\rho}) = (0.1, -3)$, where batch size is 64, when training on **CPU**.

Dataset batch n	Layer #1	Layer #2	Layer #3	$\sum_{i=1}^{ \boldsymbol{\sigma} } \sigma_i$
1	49.7534	50947.5308	49.7534	51047.0376
2	49.7534	50947.5308	49.7534	51047.0376
3	49.7534	50947.5308	49.7534	51047.0376

Table 7: How values of the summation of all elements within $\boldsymbol{\sigma}$ change, during 1st training epoch, when all model parameters are initialised with $(\boldsymbol{\mu}, \boldsymbol{\rho}) = (0.1, -3)$, where batch size is 64, when training on **GPU/CUDA**.

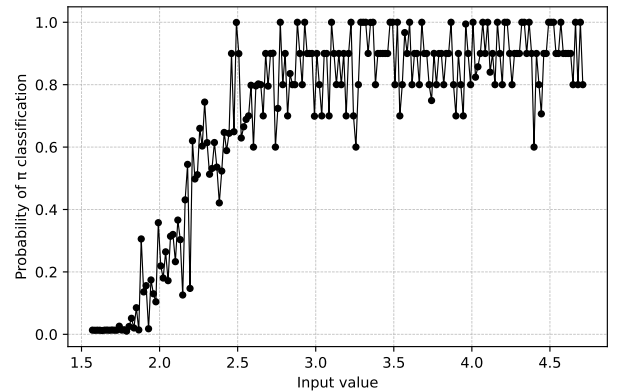
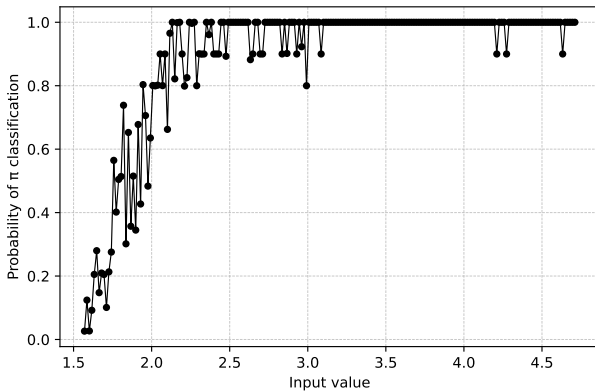
Table - (above) indicates that parameters do successfully iterate using the reparameterisation implementation. However, when the experiment is re-run on a GPU instance, $\sum_{i=1}^{|\sigma|} \sigma_i$ stagnates immediately, as shown in Table - (above). And, despite the CPU environment suggesting the possibility of the vanishing gradient problem, the GPU environment exhibits the same problem even if experiment environment is changed to *float64*.

These results show that the underlying implementations of PyTorch are not designed for reproducibility across different processor types (i.e. MIMD, SIMD). Perhaps the random seed is handled very differently. Despite the underlying PyTorch hardware-specific implementations having been inspected during Experiment #3, this was still an unexpected result.

In the tradition of science, it was then accidentally found that, when the values of (μ, ρ) were initialised differently, in that $\mathbf{w}$ and $\mathbf{b}$ were initialised with different distributions, then parameter iteration can be observed on GPU.

Dataset batch n	Layer #1	Layer #2	Layer #3	$\sum_{i=1}^{ \sigma } \sigma_i$
1	762.2622	780556.4722	762.2622	782080.9966
2	762.2617	780556.4722	762.2622	782080.9961
3	762.2613	780556.4721	762.2622	782080.9956
4	762.2613	780556.4721	762.2622	782080.9956

Table 8: How values of the summation of all elements within σ change, during 1st training epoch, when model parameters are initialised with $(\mu_w, \rho_w) = (0.1, 0.1)$, $(\mu_b, \rho_b) = (-3, -3)$, where batch size is 64, when training on GPU/CUDA.



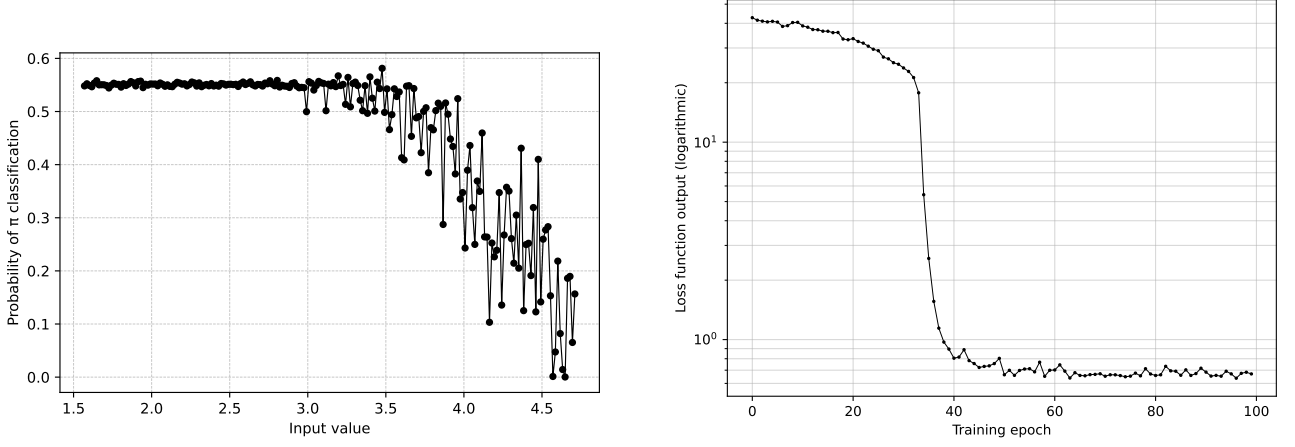


Figure 21: Three line graphs to show probability distributions generated, of n , $n \in \{\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2} \mid n \in \mathbb{R}\}$ being classified as π , when training reparameterised layers, where prior $(\mu_w, \rho_w) = (0.1, 0.1)$, $(\mu_b, \rho_b) = (-3, -3)$, batch size is 16, and KL-divergence is not a part of the loss function. Training epoch sizes are as follows, from top-left to bottom: $\{10, 20, 100\}$. Training time was 381s for 20 epochs, 1903s for 100 epochs. The loss function output graph corresponds to the training process for the related probability distributions.

The loss function graph, seen in Figure - (above), indicates stagnation after ~ 50 epochs. Combined with the probability distribution generated after 100 epochs, this suggests that the reparameterisation training sequence has large difficulties fitting the underlying parameter distribution. Given the gradient at epoch ~ 34 is a maximum (i.e. $\frac{dL}{dt} \approx -\infty$, if modeled with a mathematical function), it was thought to investigate what would happen if training epochs were reduced to 40.

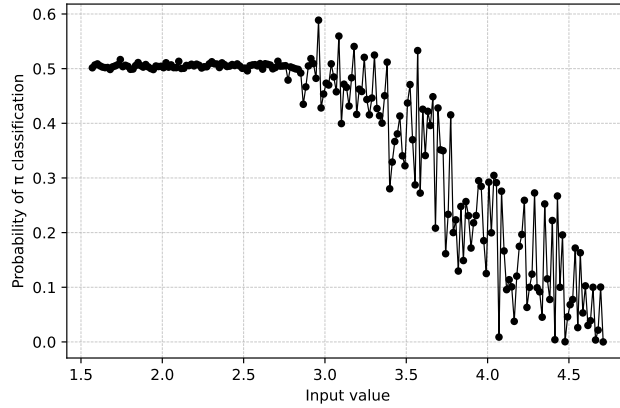


Figure 22: Line graph to show probability distributions generated, of n , $n \in \{\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2} \mid n \in \mathbb{R}\}$ being classified as π , when training reparameterised layers, where prior $(\mu, \rho) = (0.5, -5)$, batch size is 16, and KL-divergence is not a part of the loss function, for 40 training epochs.

It was next considered that changing the optimiser away from stochastic gradient descent, which uses a fixed learning rate, to the Adam optimiser (as seen in Bayesian-Torch library),

which uses a dynamic learning rate, may suit the more noisy context of reparameterisation training.

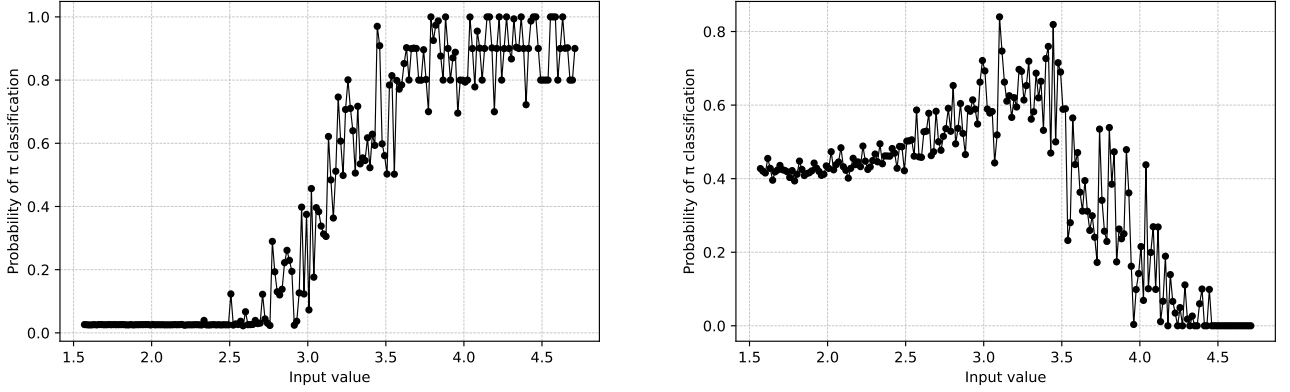


Figure 23: Two line graphs to show probability distributions generated, of n , $n \in \{\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2} \mid n \in \mathbb{R}\}$ being classified as π , when training reparameterised layers, where prior $(\mu_w, \rho_w) = (0.1, 0.1)$, $(\mu_b, \rho_b) = (-3, -3)$, batch size is 16, and KL-divergence is not a part of the loss function. Training epoch sizes are as follows, from left to right: $\{34, 35\}$. Training time was 704s for 35 epochs. Each probability is based upon 10 samples of $\hat{y}$. Parameters iterated via Adam optimiser.

The last trial was then to enable the KL divergence loss function, which is based on summing every KL divergence between the prior parameters and the approximation function that is developed, and then taking the mean. Unfortunately this did not improve the overall modeling capabilities of the neural network, the loss functions were values of equal ratios. Ultimately, the effect the KL-divergence will have on backpropagation is that the approximation function will drift towards the prior, and this may not be a particularly useful outcome. However, to retain some theoretical coherence, the KL divergence can be weighed as a ratio of the total loss function, for example a ratio of 1:9. One issue is that this ratio is a whole new hyperparameter, also ambiguous regarding exactly how to initialise successfully. For example, even if weighed as a ratio of 1:99, the KL-divergence loss still decreases at a significantly more rapid rate.

Training epoch	KL divergence loss	Total loss
1	233.443	257.037
2	6.4	30.470
3	0.2281	24.539235
4	0.0353	24.985450
5	0.0247	24.943174

Table 9: Loss function output data after multiple training epochs, when running validation sequence, when KL-divergence loss function and binary cross-entropy loss function are weighed equally, where prior $(\mu_w, \rho_w) = (0.1, 0.1)$, $(\mu_b, \rho_b) = (-3, -3)$, batch size is 16.

Training epoch	KL divergence loss	Total loss
1	22.8653	63.492329
2	18.9044	61.543575
3	16.3657	59.221767
4	12.8557	60.522063
5	11.3779	60.955337
6	9.3089	58.659589
7	6.0682	55.609534
8	2.0557	51.501864
9	0.1729	49.672922

Table 10: Loss function output data after multiple training epochs, when running validation sequence, when KL-divergence loss function and binary cross-entropy loss function are weighed in a 1:99 ratio, where prior $(\mu_w, \rho_w) = (0.1, 0.1)$, $(\mu_b, \rho_b) = (-3, -3)$, batch size is 16.

As a result of the KL divergence dominating the total loss function, the total loss function tends to stagnate, seemingly due to the approximation function becoming anchored to the prior distribution, and therefore the neural network encounters difficulties developing predictive capabilities by developing onwards from the prior.

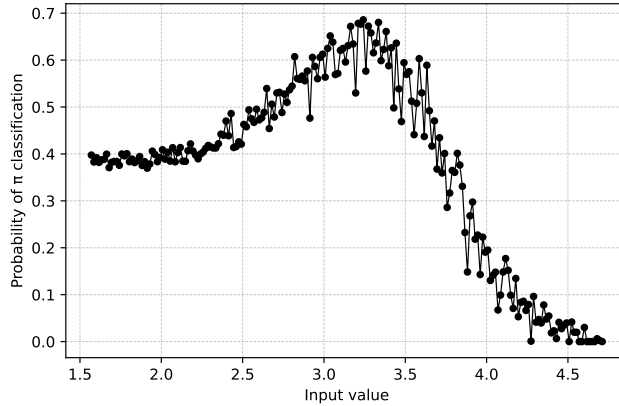


Figure 24: Line graph to show probability distributions generated, of n , $n \in \{\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2} \mid n \in \mathbb{R}\}$ being classified as π , when training reparameterised layers, where prior $(\mu_w, \rho_w) = (0.1, 0.1)$, $(\mu_b, \rho_b) = (-3, -3)$, batch size is 16, for 35 training epochs, where KL-divergence is a part of the loss function with a ratio of 1:99999. Each probability is based upon 10 samples of $\hat{y}$. Parameters iterated via Adam optimiser.

3.6 Experiment #6 - Practical CBDL

3.6.1 Purpose

At this point the difficulty of implementing reparameterisation was clear, with or without native major library support, yet the desired functionality of probabilistically scored neural network

predictions remains. As a result, it seemed worthwhile to go back to basics, by using a combination of methods which make small tweaks to a typical deterministic neural network, such as:

- Varying the weight initialisation distributions (to vary the prior)
- Utilising dropout to variate the model, or trying a variety of neuron setups (to vary the likelihood)
- Adding noise to the gradient descent procedure, a process known as Stochastic Gradient Langevin Dynamics [MCO17]

The former two are simplest, and can repurpose code from Experiment #5 (multi-sample utilisation) and Experiment #4 (parameter initialisation), whilst being just enough to fulfill the requirements of CBDL in respect to varying the likelihood and prior distributions.

3.6.2 Methodology

MC Dropout PyTorch natively supports a layer named Dropout, which takes a probability and zeros out values between layers accordingly. Though the documentation does not specify this, the Dropout layer is generally placed after an activation function (as opposed to be a replacement for a Linear layer, for example), which means there are no dimensionality complications. By default the zeroing out of values only occurs during training, however it is trivial to also trigger this functionality during utilisation.

Combinatorics One of the simplest examples would be of 3 models (likelihoods) and 3 prior distributions. Either a uniform distribution of $U(a, a)$ is suitable, or a set of random seeds with a more varied set of outputs.

Underfit To ensure that underfitting occurs, either the training epochs can be reduced, the batch size can be increased, or the training dataset can be reduced.

Confidence regions Once a probability distribution has been generated for each different initialisation combination, as a result of the PDFs being 1-dimensional it is relatively simple to integrate over a region to calculate specific confidence scores.

Measuring uncertainty To deduce the epistemic and aleatoric uncertainty, the probability distributions are to be integrated in the entirety. Note that this integrand has been reduced to the limits of $\pi \pm \frac{\pi}{2}$, but this is purely for pragmatic reasons. The analytical solution is then as follows:

$$H(X) = - \int_{\mathbf{x}} f(x) \log f(x) dx.$$

However, due to the distributions generated being approximations, instead the trapezium rule is deployed to generate the entropy metrics.

3.6.3 Results

Random seed	Loss function output
239852	0.073118
239853	0.064515
239854	0.059119

Table 11: Loss function output after 1 training epoch, when training with dropout constant $p = 0.5$, when using PyTorch’s dropout implementation with different random seeds.

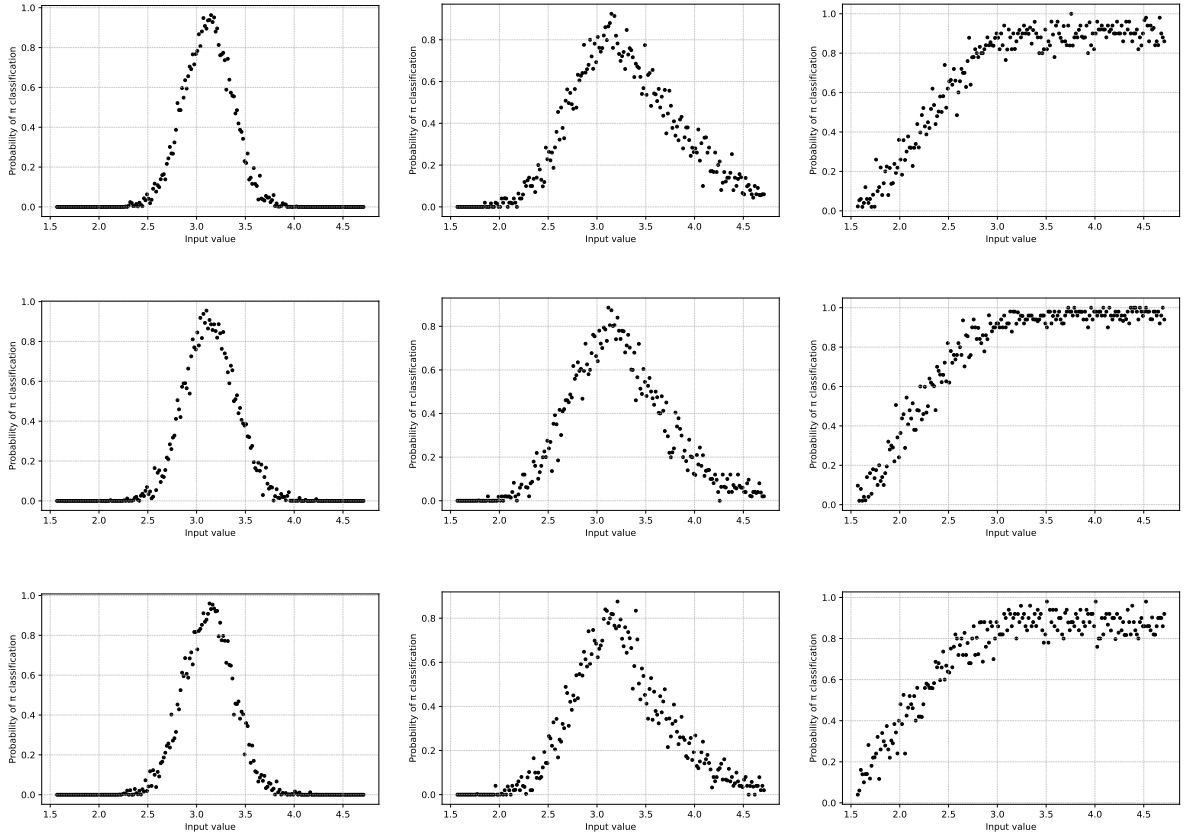


Figure 25: Line graphs to show probability distributions generated, of n , $n \in \{\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2} \mid n \in \mathbb{R}\}$ being classified as π , based on controlled model trained for 1 epoch with dropout layers, batch size is 256, where prior $U(a, a)$, $a \in \{0.25, 0.5, 0.75\}$ (one per column), and dropout layers are derived by 3 different random seeds, s , $s \in \{5453, 7876974563457, 67498896\}$ (one per row), in the case of the dropout hyperparameter being equal to 0.5, each probability being the mean of 10 samples also generated via dropout.

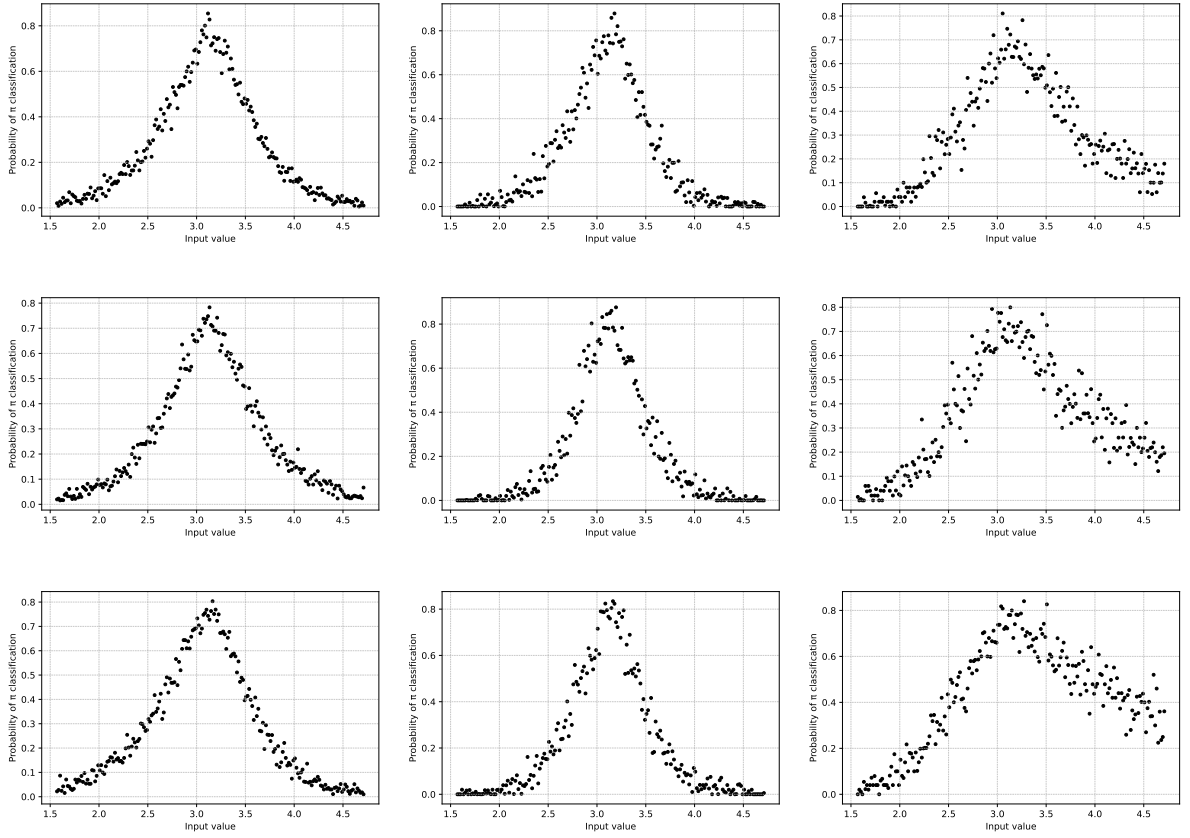


Figure 26: Line graphs to show probability distributions generated, of n , $n \in \{\pi - \frac{\pi}{2} \leq n \leq \pi + \frac{\pi}{2} \mid n \in \mathbb{R}\}$ being classified as π , based on controlled model trained for 2 epochs with dropout layers, batch size is 256, where prior $U(a, a)$, $a \in \{0.125, 0.25, 0.375\}$ (one per column), where dropout layers are derived by 3 different random seeds, s , $s \in \{5453, 67498896, 7876974563457\}$ (one per row), in the case of the dropout hyperparameter being equal to 0.8, each probability being the mean of 10 samples also generated via dropout.

		Uniform distribution $U(a, a)$		
		$a = 0.125$	$a = 0.25$	$a = 0.375$
Random seed	5453	0.7673	0.6155	0.8496
	67498896	0.8080	0.5400	0.9100
	7876974563457	0.7858	0.5560	0.9067

Table 12: Entropy values, for all combinations 3 instances of $U(a, a)$ and 3 unique random seeds.

Figure _ and _ represent a minimalist implementations of finitely generated credal sets. Figure _ depicts the values of exP_{prior} being $U(a, a)$, $a \in \{0.125, 0.25, 0.375\}$, whilst $exP_{likelihood}$ being the models generated via dropout an a set S of random seeds, where $|S| = 3$, the 3^2 combinations of resulting predictive posteriors, $P(\hat{y} \mid \theta) \cdot P(\theta \mid D)$, have then been graphed.

Uncertainty type	Value
Aleatoric	0.5400
Epistemic	0.3700
Total	0.91

Table 13: Uncertainty values, for all combinations of $U(a, a)$ where $a \in \{0.125, 0.25, 0.375\}$ and 3 random seeds s , $s \in \{5453, 67498896, 7876974563457\}$.

3.7 Conclusions

CBDL has potential to address ambiguities resulting from scaling neural networks, and scaling has been shown to instantly improve predictive capability in this context. There is also potential to further develop the metrics produced via the CBDL method, to more closely inspect the ambiguities. There are computational tribulations, however, when it comes to implementing variational inference, so as to generate the $P_{posterior}$ set, and they are not entirely clear how to overcome. In the meantime there are simpler, albeit less thorough, means of applying the CBDL method, such as for demonstration or small-scale purposes.

4 Potential further research

A potential idea for further investigation would be to try to automatically hone in on specific prior distributions, based on the clarity of different distributions generated within an ensemble trained simultaneously for n training epochs. This approach could prove to be significantly faster than simply training one specific model for n^2 training epochs, for example.

It is possible that there may be benefits from writing an entirely new optimiser, considering that major optimisers are largely written for deterministic neural networks, which is able to explore the parameter space effectively but efficiently. Alternatively, it may be worthwhile to adjust the training process of deterministic neural networks, such that the iterations made to θ can be repurposed for probabilistic neural networks, as an alternative way of exploring the parameter space.

References

- [BCKW15] Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural networks. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015.
- [CDJ⁺23] Michele Caprio, Souradeep Dutta, Kuk Jin Jang, Vivian Lin, Radoslav Ivanov, Oleg Sokolsky, and Insup Lee. On the convergence of bayesian neural networks. *arXiv preprint arXiv:2302.09656*, 2023.
- [Coz97] Fabio G. Cozman. Robustness analysis of bayesian networks with local convex sets. Technical Report CMU-RI-TR-97-06, Carnegie Mellon University, The Robotics Institute, 1997.
- [CS24] Rohitash Chandra and Arunesh Simmons. Bayesian neural networks via mcmc: A python-based tutorial. *IEEE Access*, 2024. IEEE Xplore Document 10530647.
- [FGAvdWA21] Vincent Fortuin, Adrià Garriga-Alonso, Mark van der Wilk, and Laurence Aitchison. Bnnpriors: A library for bayesian neural network inference with different prior distributions. *Software Impacts*, 8:100079, 2021.
- [GF20] Ethan Goan and Clinton Fookes. Bayesian neural networks: An introduction and survey. *arXiv preprint arXiv:2006.12024*, 2020. See Equation 17 for BNN intro.
- [GVLD⁺23] William F. Godoy, Pedro Valero-Lara, T. Elise Dettling, Christian Trefftz, Ian Jorquera, Thomas Sheehy, Ross G. Miller, Marc Gonzalez-Tallada, Jeffrey S. Vetter, and Valentin Churavy. Comprehensive comparison of julia and python for data science and machine learning. *arXiv preprint arXiv:2303.06195*, 2023.
- [HZRS15] Kaiming He, Xiangyu Zhang, Shao-hua Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. *arXiv preprint arXiv:1502.01852*, 2015.
- [JLB⁺20] Laurent Valentin Jospin, Hamid Laga, Farid Boussaid, Wray Buntine, and Mohammed Bennamoun. Hands-on bayesian neural networks – a tutorial for deep learning users. *arXiv preprint arXiv:2007.06823*, 2020.
- [KL51] Solomon Kullback and Richard A. Leibler. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951. Historical context: NSA/Dr. Richard A. Leibler.
- [MCO17] Gaétan Marceau-Caron and Yann Ollivier. Natural langevin dynamics for neural networks. *arXiv preprint arXiv:1712.01076*, dec 2017.
- [Min00] Thomas P. Minka. Estimating a dirichlet distribution. Technical report, Microsoft Research, 2000.

- [SS06] Devinderjit Sivia and John Skilling. *Data Analysis: A Bayesian Tutorial*. Oxford University Press, 2nd edition, 2006. See figure 2.2 for coin flip graphs related to deducing coin bias. See equation 3.19 for a two-parameter analytical solution, equation 3.29 for the generalisation to any quantity of parameters.
- [Tom14] Carlo Tomasi. The dirichlet distribution. Course Notes, CompSci 527, Duke University, 2014.

5 Appendix

[illegible]

[illegible]